Peer-to-Peer Networking and Applications (2025) 18:161
https://doi.org/10.1007/s12083-025-01947-4

A feature selection-driven machine learning framework for anomaly-based intrusion detection systems

Emre Emirmahmutoğlu[1] · **Yılmaz Atay**[2]

Received: 9 December 2024 / Accepted: 28 February 2025 / Published online: 28 April 2025
© The Author(s) 2025

Abstract

In light of rapid technological developments, a marked rise in global internet usage has contributed to increased sensitive data flow across networks. This increase leads to the diversification of malicious attacks and makes cyber security requirements more evident. In order to ensure network security, intrusion detection systems stand out as an essential component. Intrusion detection systems detect suspicious and malicious activities over network traffic, allowing network administrators and experts to monitor current threats continuously. In anomaly-based systems, machine learning approaches are applied to identify abnormal attempts in network traffic. This study presents a feature selection framework for anomaly-based attack detection systems by combining machine learning and heuristic algorithms. This proposed study aims to improve the performance of IDSs in terms of both time and attack detection by selecting features with heuristic approaches. In the proposed approach, PSO, FPA, DE feature selection methods and LR, DT, RF, KNN, NB, GB, LDA, QDA, AdaBoost, and NN machine learning algorithms are used to perform anomaly-based comparative analyses on KDDCup99, NSL-KDD, UNSW-NB15, CSE-CIS-IDS2018 datasets. Analyses conducted on these datasets with various features demonstrated that models employing feature selection achieved an approximate two-hundred-percent improvement in time efficiency compared to models that did not utilize feature selection. It has been determined that DE, PSO, and FPA, which are used for feature selection, provide high-accuracy outputs when combined with different classifiers. When the analysis results are assessed according to the specified criteria, the highest F1-Score values achieved are as follows: 0.9972 for the DE method in GB, 0.9969 for the PSO method in GB, and 0.9948 for the FPA method in GB, on the KDD CUP 99 dataset. In the NSL-KDD, used as the second dataset, the DE method achieved a score of 0.9713 in GB, the PSO method reached 0.9112 in DT, and the FPA method obtained 0.9894 in RF, respectively. In the third dataset, UNSW-NB15, the DE method achieved a score of 0.9507 in DT, the PSO method reached 0.9068 in DT, and the FPA method obtained 0.8924 in NN. Finally, in the CSE-CIC-IDS2018 dataset, the highest scores achieved using the RF algorithm were 0.99986 for the DE method, 0.99989 for the PSO method, and 0.99987 for the FPA method, based on feature selection. The obtained results underscore the critical role of dataset generation processes and network traffic dynamics in enhancing the performance of intrusion detection systems. Additionally, the significance of feature selection was highlighted. These findings offer valuable insights and present opportunities for further advancements in future research.

Keywords Cyber Security · Intrusion Detection System · Machine Learning · Feature Selection · Computer Networks · Classification

1 Introduction

Due to the increasing internet usage, the volume of important, sensitive, and confidential personal and commercial data traveling over the global network has grown rapidly. The ever-changing nature of network attacks and the intensification of data sharing between computing devices have made the provision of network and information security a more critical requirement. By 2022, global IP traffic is

This article is part of the Topical Collection: *Special Issue on 2 - Track on Security and Privacy*
Guest Editor: Rongxing Lu

Extended author information available on the last page of the article

projected to triple, corresponding to an annual growth rate of 26 percent [1]. While per capita net business traffic was measured at 16 GB per month in 2017, this value is expected to reach 50 GB in 2022 [1]. Today, it is seen that this traffic has reached even higher levels than predicted. The tools used to exchange information, collect data, and perform monitoring activities over the network expand the attack surface. Attackers try to breach network security by using vulnerabilities that can disrupt systems and affect data confidentiality and access. Therefore, cyber security stands out as an area that offers critical measures to protect against cyber-attacks and reduce costs. In this context, the Intrusion Detection System (IDS), one of the key components of the security architecture, functions as a tool that can identify different types of intrusions. IDS allows network administrators to monitor current threats by observing network traffic and detecting suspicious or malicious activities and policy violations. This requirement has led to the necessity of establishing an effective IDS system that can protect networks from attacks. Over time, IDSs have evolved to safeguard network traffic from malicious activities and security breaches. To address the growing complexity of network attacks, next-generation firewalls have been developed by integrating IDS modules that utilize signature analysis, behavioral analysis, and the detection of malicious activities. An IDS is a system designed to alert users when unusual or suspicious activities are identified within a network. The primary goal of an IDS is to secure the system by identifying intruders or malicious actions that may compromise security [2].

IDSs are generally categorized as host-based or network-based, depending on their deployment environment, location, and data sources. A host-based IDS operates on individual devices, monitoring data packets to detect suspicious activity or breaches of security policies [3]. However, this approach has limitations, as each device requires a separate IDS installation, which can lead to performance degradation due to the additional processing load. Conversely, a network-based IDS is deployed across the network and aims to detect malicious actions by monitoring, capturing, and analyzing traffic from multiple sources (Rao Umesh [4]). For intrusion detection, IDSs primarily rely on two analysis methods: anomaly-based and exploit-based (or signature-based) detection [3]. Signature-based IDSs, also known as abuse intrusion detection systems, identify attacks by matching network data with known attack signatures or patterns stored in a database. When a match is found, the system flags the activity as malicious. While this method is effective at detecting known attacks, it fails to identify new or zero-day attacks, as its detection capability is limited to the signatures available in the database.

Maintaining and updating the signature database and comparing it to network data also demands significant computational resources. On the other hand, anomaly-based IDSs, also called behavior-based intrusion detection systems, detect attacks by modeling normal system behavior and flagging deviations from this model. The advantage of this approach is its ability to identify new and previously unknown attacks. However, it often has a higher rate of false positives, which can limit its practical application. As the variety and sophistication of network threats continue to grow, intrusion detection remains a critical and dynamic area of research, driving innovations to enhance security in network and computing environments [2].

Researchers have been conducting various studies to develop effective IDS methods. In addition to signature-based approaches, many recent IDS methods in the literature use anomaly-based methods supported by artificial intelligence, i.e., machine learning (ML) and deep learning (DL) techniques, to identify intrusions. In the cybersecurity field, machine learning techniques are increasingly used to enhance the effectiveness of critical systems like IDSs, improving threat detection, reducing false positives, and adapting to evolving attacks. However, while many studies in the literature focus on improving the accuracy of machine learning models, they do not adequately address the fact that redundant and uncorrelated features in large data sets negatively affect system performance [5, 6]. This increases the computational burden and limits the overlearning and generalization capabilities of the models. Feature selection algorithms stand out as an effective tool for solving this problem. In the literature, studies on applying meta-heuristic algorithms in feature selection and intrusion detection systems on big data have gained momentum, but research in this area is still limited [7, 8]. This study highlights the ability of meta-heuristic algorithms such as FPA (Flower Pollination Algorithm), DE (Differential Evolution), and PSO (Particle Swarm Optimization) to optimize IDS performance by integrating them into the feature selection process. It aims to fill the gap in this field.

In this study, we aim to detect intrusions in data sets using machine learning algorithms. KDD CUP 99, NSL-KDD, UNSW-NB15, and CICIDS2018 datasets, which are the subject of a wide range of studies in the literature, were used for this purpose. Meta-heuristic algorithms such as particle swarm optimization, flower pollination, and differential evolution algorithms were used for feature selection. Feature selection based on machine learning algorithms is performed on the datasets, and classification results without feature selection are presented comparatively. It is aimed to present a comparative analysis of machine learning algorithms using the datasets.

In the following sections of the paper, the literature review, the general framework of intrusion detection systems, the materials and methods used, and the experimental studies and results are presented in detail. The literature review offers a comprehensive overview of previous

research and key developments in the field, establishing the context for the current work. The general framework of IDS introduces the foundational concepts, architectures, and approaches that guide the detection of network intrusions. The materials and methods section describes the datasets, algorithms, and tools employed, clarifying how the experiments were conducted. Finally, the experimental studies and results section presents the findings of the experiments, offering in-depth analysis and discussion of the outcomes that contribute to the advancement of IDS technology and its applications.

2 Literature Review

Within the scope of the literature review, general research on intrusion detection systems has been carried out, and information has been obtained from different databases about the emergence of intrusion detection systems, working principles, usage areas according to their structure, attack categories, and attack types.

Based on the existing studies, the focus is on the impact of intrusion detection systems on cyber security. The scope of the research is narrowed down to network and anomaly-based intrusion detection systems. In this context, academic and open sources were consulted, and the focus was on the existing data sets in the literature and the intrusion detection studies with machine learning algorithms on them.

Salek et al. conducted a study utilizing the DE algorithm in combination with a neural network classifier to optimize feature selection and enhance the effectiveness of intrusion detection systems. Their approach was evaluated using the KDDCUP99 dataset, yielding a notable accuracy rate of 96% [9].

Popoola and Adewumi [10] employed the DE algorithm alongside a Decision Tree (DT) classifier to refine feature selection in their efforts to improve intrusion detection systems. Their method was tested on the NSL-KDD dataset, achieving an accuracy rate of 89% [10].

Hajisalem and Babaie developed a hybrid classification approach by integrating the Artificial Bee Colony (ABC) and Artificial Fish Swarm (AFS) algorithms [11]. This method was assessed using the NSL-KDD and UNSW-NB15 datasets. To enhance efficiency, the Correlation-Based Feature Selection (CFS) technique and Fuzzy C-Mean Clustering (FCM) were employed to partition the training dataset and eliminate irrelevant or redundant features. Based on the selected features, the Classification and Regression Tree (CART) algorithm was applied to train the model and generate if–then rules for distinguishing between normal and abnormal records. The evaluation results demonstrated a remarkable accuracy of 99.9%, a false alarm rate of 0.01%, and a detection rate of 99.99%. Additionally, the computational overhead, complexity, and execution time of the proposed method were found to be comparable with existing methodologies.

Selvakumar and Muneeswaran introduced a model employing the Mutual Information Firefly Algorithm (MIFA) as a wrapper for feature selection [12]. Bayesian Network (BN) and C4.5 classifiers were utilized to evaluate the model's performance on the KDD CUP 99 dataset. Experimental results indicated that the proposed approach significantly improved accuracy and reduced false alarm rates. The model achieved accuracies of 99.98%, 63.85%, 98.73%, 17.24%, 99.95%, 93.42%, 97.83%, and 68.97% for C4.5 and BN classifiers across various attack categories. False positive rates (FPR) were reported as 0.01 for DoS, 0.04 for Probe, 0.04 for R2L, and 0.04 for U2R attacks. Despite its effectiveness, the model was noted for its high computational demands.

Mazini et al. proposed a hybrid approach combining the ABC algorithm and AdaBoost [13]. The ABC algorithm was employed for feature selection, while AdaBoost classified the selected features. When tested on the NSL-KDD and ISCXIDS2012 datasets, the method achieved an accuracy of 98.9%. The proposed technique was also benchmarked against other studies on these datasets.

Adhi Tama et al. introduced a feature selection approach utilizing two-level classifier ensembles [14]. Techniques such as the ant colony algorithm, particle swarm optimization, and genetic algorithms were applied to reduce the feature size of the training datasets. The REPTree algorithm was employed to identify features that optimize classification performance. Subsequently, a two-level ensemble of classifiers, incorporating rotation forest and bagging methods, was utilized. The proposed system was tested on the NSL-KDD and UNSW-NB15 datasets, achieving accuracy rates of 85.8% and 91.3%, respectively. The study demonstrated that employing advanced techniques with fewer features could enhance classification accuracy.

Meftah et al. proposed a two-stage anomaly-based intrusion detection system using the UNSW-NB15 dataset [15]. In the first stage, recursive feature elimination and random forests were applied to identify the most critical features. In the second stage, support vector machines (SVM), gradient boosting machines (GBM), and logistic regression were used for binary classification to detect abnormal traffic. The SVM classifier achieved an accuracy of 82.1%, and the hybrid two-stage classification approach improved the overall accuracy to 86.04%.

Kanimozhi and Jacob achieved comparable results by applying Artificial Neural Networks (ANN) to the CSE-CIC-IDS2018 dataset [16]. Their model demonstrated a high accuracy of 99.9% and excellent recall and precision rates while maintaining a low false alarm rate. However, the study noted that the model focuses on a limited number of

attack types and requires substantial time for training and preprocessing.

Gao et al. proposed an adaptive ensemble learning model that integrates multiple DTs with an adaptive voting mechanism [17]. The model was evaluated using the NSL-KDD dataset, yielding adaptive accuracy rates ranging between 84.23% and 85.2%.

Zhou et al. introduced a novel IDS architecture combining feature selection and ensemble learning techniques [18]. They utilized the Correlation-Based Feature Selection Bat Algorithm (CFS-BA) to identify the optimal feature subset. Subsequently, Random Forest (RF) and C4.5 classifiers were combined for intrusion detection. This model was tested on the CIC-IDS-2017, AWID, and NSL-KDD datasets, achieving a superior performance accuracy of 99.81% compared to other methods.

Kim et al. proposed a model that achieved an accuracy of 99% by utilizing Convolutional Neural Networks (CNN) on the CSE-CIC-IDS2018 dataset [19]. However, the approach was focused solely on DoS attacks and required considerable computational resources.

Alazzam et al. developed a pigeon-inspired optimizer (PIO) for feature selection, aiming to minimize the number of features required to build an effective IDS while maintaining high accuracy and low false positive rates [20]. The PIO algorithm significantly reduced the feature set in the UNSW-NB15, NSL-KDD, and KDD CUP 99 datasets, lowering the number of features from 49 to 5, 41 to 5, and 41 to 7, respectively. The experimental results demonstrated that the proposed method preserved high accuracy and true positive rates while reducing training time.

Devan and Khare developed an XGBoost-DNN model for intrusion detection, where the Extra Gradient Boosting (XGBoost) technique was employed for feature selection, and the Deep Neural Network (DNN) was trained using the NSL-KDD dataset to detect network intrusions [21]. The performance of this model was compared to existing classifiers such as Naive Bayes, logistic regression, and support vector machines. The results demonstrated that the DNN model outperformed the other techniques, achieving a classification accuracy of 97%.

Nazir and Khan proposed a wrapper-based feature selection method called TS Random Forest, combining Tabu Search (TS) as a search technique and Random Forest as the learning algorithm for intrusion detection systems [22]. The model was evaluated using the UNSW-NB15 dataset. Compared to previous feature selection methods such as Chi-square and Pearson correlation, the proposed model achieved an accuracy of 83.12% and a false positive rate of 3.7%. However, the model did not address the class imbalance issue within the dataset, which could potentially affect the classifier's accuracy by increasing the false positive and misclassification rates.

Liu et al. proposed an IDS model that utilizes the Adaptive Synthetic (ADASYN) oversampling technique and the LightGBM ensemble [23]. ADASYN was employed to address the issue of data imbalance, while LightGBM was implemented as the classifier to reduce the computational complexity of model training and intrusion detection time. The model was tested on the UNSW-NB15, NSL-KDD, and CICIDS2017 datasets. Experimental results revealed that LightGBM achieved accuracy rates of 83.98%, 89.79%, and 99.86% for these datasets, respectively. Furthermore, applying ADASYN oversampling improved the detection rate for minority classes, leading to overall detection accuracies of 85.89%, 92.57%, and 99.91%. However, the model still exhibited a relatively high false positive rate, indicating that further improvements are needed to address this issue.

Thilagam and Aruna introduced a Recurrent Convolutional Neural Network (RC-NN) for intrusion detection, integrating CNN, Long Short-Term Memory (LSTM), and Ant Lion Optimization (ALO) algorithms [24]. The proposed approach was evaluated using the DARPA and CSE-CIC-IDS2018 datasets and compared with existing methods. The model demonstrated a significant reduction in error rate to 0.0012% and achieved a higher classification accuracy of 94%. As a result, this method outperformed previous research in terms of accurately identifying each type of attack.

Disha and Waheed explored various machine learning models, including decision tree, multilayer perceptron (MLP), AdaBoost, Gradient Boosting Tree (GBT), Gated Recurrent Unit (GRU), and LSTM, for the binary classification task of detecting network intrusions [25]. The models were evaluated using the UNSW-NB15 and Network TON IoT datasets. Feature selection was performed using the Gini Impurity Based Weighted Random Forest (GIWRF) method, identifying 20 critical features from the UNSW-NB15 dataset and 10 from the Network TON IoT dataset. GIWRF used Gini impurity to split trees and adjusted weights for the two unbalanced data classes, helping the learning algorithm better grasp class distribution. Experimental results indicated that the DT model outperformed the other machine learning models when using the feature selection method. However, the study did not address multiclass classification or temporal complexity analysis.

Ahmad et al. [26] proposed an AdaBoost-based model for network intrusion detection using the UNSW-NB15 dataset, comparing it with MLP and SVM models (I. [26]). The proposed method achieved an impressive accuracy of 99.3% and could detect various types of network attacks. Similarly, Qureshi et al. [27] introduced a novel random neural network-based IDS using the NSL-KDD dataset, which achieved an accuracy of 94.5% [27].

Mohsenabad and Tut focused on IDSs as a vital solution to combat the rising number of cyberattacks in computer

network security. In their study, they applied ABC, FPA, and Ant Colony Optimization (ACO) for feature selection, resulting in models that achieved accuracy rates of 99.0% (ACO), 98.7% (FPA), and 98.6% (ABC). Their findings offer valuable insights for network administrators in choosing suitable machine learning algorithms utilizing the CSE-CIC-IDS2018 dataset [28].

Zare et al. proposed a Network-based Anomaly Detection System (NADS) to identify data manipulation attacks on Modbus/TCP-based SCADA systems. Their approach employed a sequence-to-sequence autoencoder using LSTM units, with an embedding layer, teacher coercion technique, and attention mechanism. The model was trained and evaluated using the SWaT dataset, which corresponds to a scaled water treatment plant. The proposed model successfully detected 23 out of 36 attacks, outperforming two existing NADS with an improvement of 0.22 for simple attacks [29].

Saheed et al. introduced a novel lightweight transfer learning method utilizing ResNet50-CNN1D for intrusion detection in cyber-physical systems. The Adaptive Gradient (Adagrad) optimizer minimizes the loss function by adjusting the network weights. The performance of the proposed ResNet50-1D-CNN model was evaluated using the UNSW-NB15 dataset and a control system dataset called HAI. The results on the HAI dataset demonstrated an accuracy of 97.32%, recall of 98.41%, precision of 97.09%, and an F1-score of 96.32%. For the UNSW-NB15 dataset, the model achieved 99.89% accuracy, 99.09% recall, 98.70% precision, and 98.01% F1-score [30].

Habeeb and Babu focused on NIDS in wireless communication of IoT devices. In their study, they used a Whale Optimization Algorithm (WOA) and Genetic Algorithm (GA) hybrid for feature selection and machine learning algorithms for classification on the BoT-IoT 2020 dataset. Their proposed model achieved 99.5% accuracy rate and reduced the false alarm rate (FAR) [31].

Yuan et al. proposed a DL-based IDS (DL-IDS) architecture by combining ML and DL models. As a result of their studies on NSL-KDD and CICIDS2018 datasets, they obtained close and better results compared to DL-IDS with existing methods. The proposed method is characterized by low resource management [32].

Truong and Le proposed the method MSecureChain for IDS with machine learning and blockchain applications in the security of Metaverse platforms. When the proposed model was analyzed on the CIC-IDS2017 dataset, it showed that about 99% accuracy can be achieved for intrusion detection [33].

These evaluations of the literature review strengthen the foundation of the proposed approach. The next section provides a detailed explanation of the general framework of Intrusion Detection Systems (IDS), outlining their key components and operational structure. This study focuses on the binary classification process, while the multi-class classification problem is planned to be addressed in future research. The primary objective is to evaluate the effectiveness of heuristic-based feature selection methods in IDS rather than classification performance based on the number of classes.

3 General framework of IDS

This section outlines the positioning of Intrusion Detection Systems (IDSs) in the existing literature, as well as their integration into real-time network infrastructures. It also discusses how administrators utilize these systems for effective monitoring and security management. An anomaly can be defined as an observation that significantly deviates from existing patterns, prompting suspicion that it may have been caused by an external tool or individual [34]. It refers to actions that disrupt the security of networks and systems, violating the foundational principles of the computer security model—confidentiality, integrity, and authentication. Anomalies are typically a result of breaching security mechanisms, gaining unauthorized access, or executing attacks within or outside the network. IDSs serve as a defense against these threats by continuously monitoring data from various sources [35]. The primary functions of IDS are as follows:

Monitor network traffic,
Analyze data collected from network traffic,
Identify attack types or signatures,
Storing attack types or signatures in a database,
Generating alerts when any anomaly or signature is detected

Figure 1 illustrates the components of IDS, which rely on monitoring network traffic and collecting data from network packets. Attackers execute network assaults by injecting malicious code or analyzing network packets to gather sensitive information. These attacks may target either the server responsible for managing network operations or the host system actively performing network activities. Additionally, attackers may exploit vulnerabilities in the system. In essence, advanced techniques like machine learning and deep learning enhance IDS capabilities, allowing for more innovative detection of network threats.

The components of an IDS, the framework of which is given in Fig. 1, are as follows: monitoring network traffic, data collection, data analysis, identifying and storing signature/attack types, and alert generation. These components are briefly explained. “Monitoring network traffic” is crucial for capturing packets that contain vital information about the network. Each network packet consists of a header and

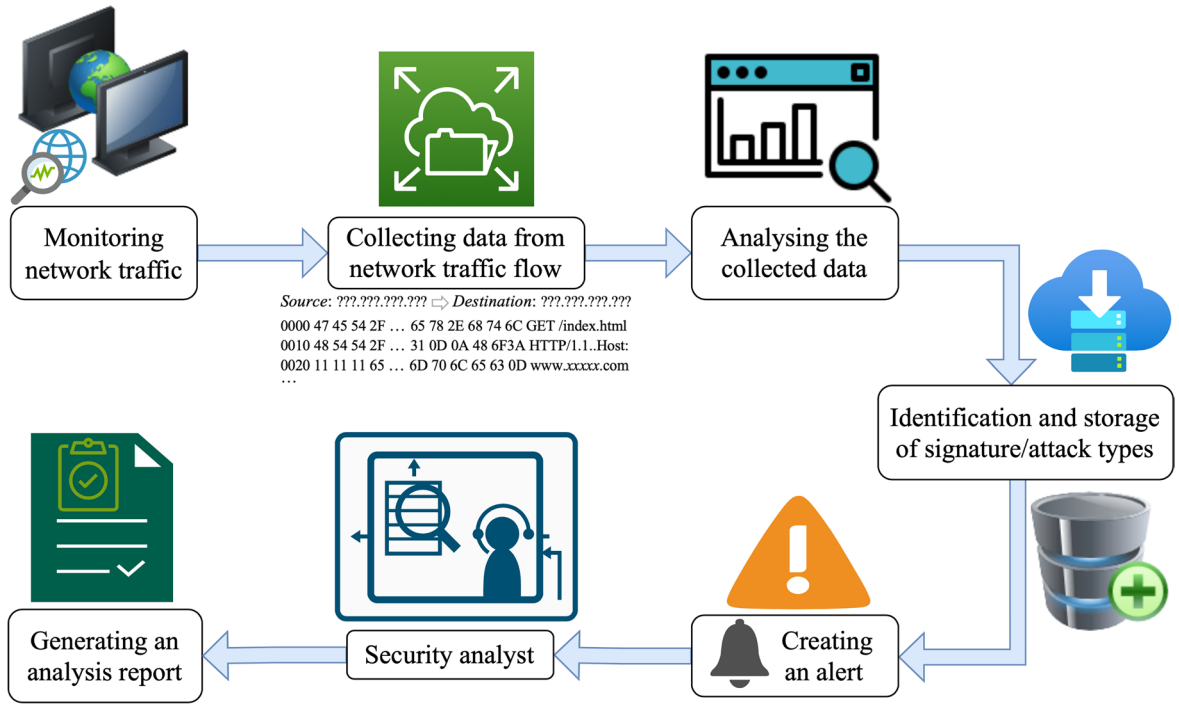


Fig. 1 Components of intrusion detection systems

a payload, both of which hold valuable data that can be analyzed to detect potential threats. In addition, network flows are examined to identify patterns that could be exploited in an attack. As a result, intrusion detection datasets often incorporate both packet-level and flow-level data to classify attacks effectively. "Data collection" involves gathering information about the target system, where potential attacks may occur. This can be achieved using network commands or tools. For example, packet-level details can be obtained by sniffing network traffic with tools like Wireshark, while server and host information, such as domain names, can be retrieved using commands like *nslookup*. "Data analysis" focuses on inspecting network packets to uncover sensitive information. Attacks such as R2L (Remote Locale) can occur by compromising a system and gaining unauthorized access. Techniques like packet sniffing to steal credentials or injecting malicious software such as Trojans for remote access are common attack vectors. These vulnerabilities are more likely to be exploited if the target system has open ports. Once packet data has been analyzed, the next step is to identify known attack signatures or patterns from new threats (identifying and storing signature/attack types), including exploits of existing vulnerabilities that could be used to initiate insider attacks. These signatures and attack patterns are stored in a database, enabling security administrators to detect and respond to anomalous activity promptly. Finally, after detecting a known attack signature or pattern, the system triggers an alert or alarm to notify the security administrator (alert generation). This alert serves as an indication of potential intrusion or abnormal behavior, allowing for swift response and mitigation of threats.

4 Material and method

This section outlines the materials and methods, offering a comprehensive framework for the research methodology. It provides a summary description of the datasets, including their sources, preprocessing steps, and features. Furthermore, the section elaborates on the feature selection techniques implemented to identify the most relevant variables, thereby optimizing the model's performance and interpretability. Additionally, it presents an in-depth discussion of the machine learning algorithms utilized, highlighting their configurations, training procedures, and evaluation metrics. These methodological details collectively form the foundation for the analysis and ensure the robustness and validity of the findings.

4.1 Datasets

To evaluate the performance of any IDS, it is essential to have a standardized dataset that facilitates the comparison of various classifiers. For instance, the DARPA-98 dataset was developed by the MIT Lincoln Lab in 1998 as part of

DARPA-funded projects. Over the past two decades, this dataset has been widely used to assess IDS models. However, several issues were identified in the dataset, including the presence of duplicate records, imbalance in the training and test data, and the inclusion of synthetic traffic [36].

To address these limitations, ongoing research has focused on creating more refined versions of the dataset, such as the NSL-KDD dataset [37]. Various organizations have since developed different datasets for the same purpose, with continuous improvements over the years. The datasets utilized in this study are described below [37].

4.1.1 KDD CUP 99

Since its creation in 1999, the KDD CUP 99 dataset has been one of the most commonly used resources for evaluating anomaly detection methods (KDD [38] Data, 2024). Stolfo et al. developed this dataset from data collected during the DARPA-98 IDS evaluation program [39]. Each record in the KDD dataset includes 41 features, with samples labeled either as normal or as an attack. The simulated attacks are classified into four categories: DoS (Denial of Service), U2R (User to Root), R2L (Remote to Local), and Probe.

4.1.2 NSL-KDD

Research on the KDD Cup 99 dataset has revealed several significant issues that negatively impact the performance of systems, leading to poor predictions by anomaly detection methods. To address these challenges, a new dataset called NSL-KDD was introduced. This dataset includes carefully selected records from the KDD dataset (NSL-KDD Datasets Research, 2024). One key advantage of the NSL-KDD dataset is that it eliminates redundant and duplicate records in both the training and test datasets, which helps reduce the number of false positives in classification results.

The training dataset includes 21 types of attacks, which are different from the 37 types found in the test dataset. The attacks in the training dataset are considered known, while those in the test dataset are new attacks, meaning they were not present in the training data. The attack types are classified into DoS, U2R, R2L, and Probe (NSL-KDD Datasets Research, 2024).

4.1.3 UNSW-NB15

The effectiveness of network IDS is determined by their ability to detect attacks, which requires a comprehensive dataset containing both normal and abnormal behaviors [40]. The KDDCUP 99 and NSL-KDD datasets are widely used to evaluate network-based IDS performance. However, these datasets have certain limitations: the KDDCUP 99 dataset contains a large number of redundant records, while the NSL-KDD dataset fails to replicate modern attack scenarios.

These issues have posed significant challenges for cybersecurity research groups, such as the two teams at the Australian Center for Cyber Security (ACCS), as well as other researchers globally. In response, the ACCS developed the UNSW-NB15 dataset, which combines real, modern, normal behavior with simulated attack activities. This dataset contains 49 distinct features, and the simulated attacks are classified into categories such as Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms [41].

4.1.4 CSE-CIC-IDS2018

Datasets, including CSE-CIC-IDS2018 dataset [42], are specifically designed to train predictive models for anomaly-based intrusion detection in network traffic [43]. CSE-CIC-IDS2018 is an extension of the ISCXIDS2012 dataset, which includes both normal and abnormal network traffic collected over seven days at the University of New Brunswick [43]. However, the ISCXIDS2012 dataset had limitations, such as the lack of essential protocols like HTTPS and unrealistic attack distributions. The CIC and ISCX teams developed the CIC-IDS2017 dataset in 2017 to address these issues. The CSE-CIC-IDS2018 dataset, released in 2018, follows a similar structure to CIC-IDS2017, but it introduces several improvements, such as addressing class imbalance and incorporating a broader variety of attacks from a wider network. This dataset contains 80 features and covers various attack categories, including brute force, Botnet, DoS, and DDoS [42, 44].

4.2 Proposed model

This section focuses on implementing the IDS architecture, the core subject of this research, to classify data obtained from network flow. The overall framework of the IDS architecture is illustrated in Fig. 1. A critical component of this framework is the feature selection step, which follows data preprocessing, as depicted in Fig. 2. Feature selection plays a vital role in enhancing the performance of classification models by eliminating irrelevant or redundant features and improving efficiency and accuracy. Meta-heuristic approaches are employed to systematically select and reduce features within datasets, optimizing overall performance evaluation. These approaches efficiently explore the high-dimensional feature space and identify the most informative features that contribute to accurate classification. Each feature selection method is individually considered and tested

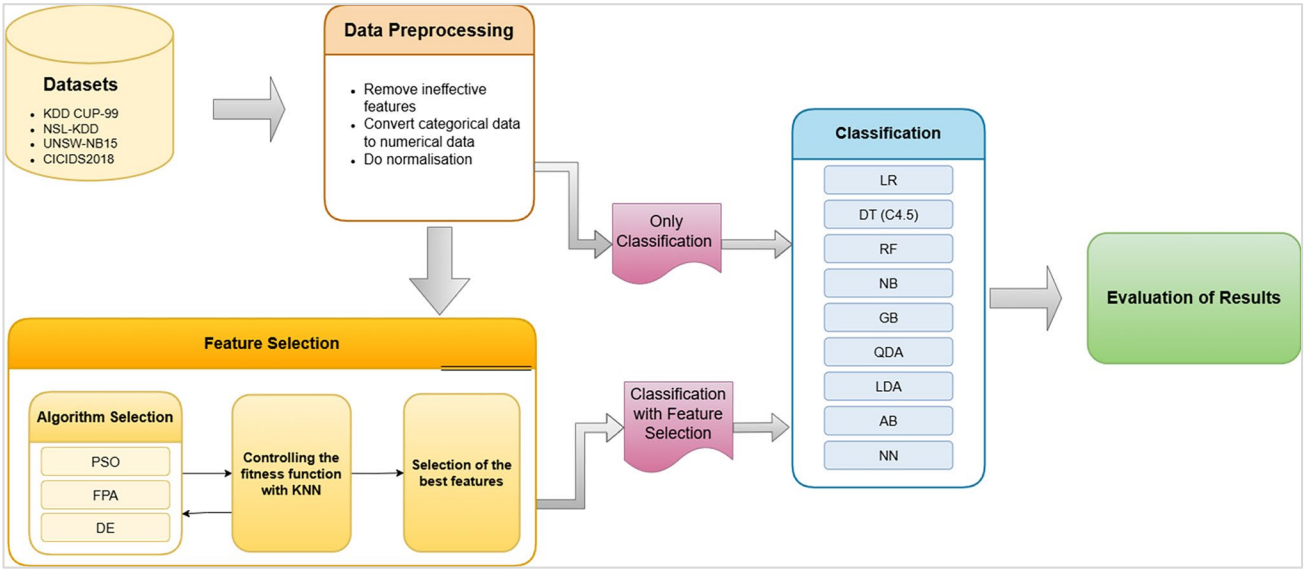


Fig. 2 Flowchart of the proposed model

to assess its impact on classification performance in this study. The most effective approaches to increase IDS efficiency were identified by comparing different methods. The following sections provide a detailed description of these feature selection techniques, highlighting their mechanisms and contributions to performance enhancement.

From the datasets used in the proposed model, clusters are first randomly determined for feature selection at a rate of ɗ %. After the analyses performed in this study, ɗ was defined as 20. Thus, while 20% of the data is used in the feature selection process, the remaining 80% constitutes the training and test set in the classification process. These data sets are read by the application respectively. The data is then preprocessed and normalized. In the preprocessing stage, features such as sequence number, attack category, etc., that will not affect the result are removed, and categorical data such as protocol, service, etc., are converted into numeric data. The conversion process is performed using the label encoding technique. Normalization is obtained by subtracting the average of the values from the value of each feature in the data set and dividing by the standard deviation. The normalization process is performed using the standard scaler method. The standard scaling method converts the mean of each feature in the dataset to 0 and the standard deviation to 1. This process reduces the impact of different scales on the features, allowing machine learning models to perform better. After this process, feature selection is performed on the data using one of the meta-heuristics: particle swarm optimization, differential evolution, and flower pollination algorithms. The classification process is started with the selected features. After feature selection, the k-fold cross-validation method was used in the classification process. This study takes the k value as five for each algorithm. Logistic regression, decision tree, random forest, linear discriminant analysis, linear discriminant analysis, quadratic discriminant analysis, adaboost, k-nearest neighbor, naive bayes, gradient boosting and neural networks machine learning algorithms are applied respectively in the model with the selected features and all the results obtained are written to the log file. The model repeats this process ten times for each data set.

4.2.1 Feature selection

This section explains the feature selection step following the data preprocessing step of the general flow given in Fig. 2. Focused on feature selection, which plays an essential role in improving the performance of classification methods. Meta-heuristic approaches are used to select and reduce datasets' features to optimize the performance evaluation. Each feature selection approach is discussed and tested separately. These methods, designed to explore the feature space efficiently, are described below in detail.

Particle swarm optimization The Particle Swarm Optimization (PSO) algorithm, introduced by R. Eberhart and J. Kennedy [45], has found widespread application in various engineering fields. Inspired by the social behaviors of animals like birds and fish, PSO simulates the collective movement of these creatures as they seek food sources (Y. [46]). The algorithm relies on individual experiences and the shared knowledge of the group to find optimized solutions. Birds, for example, follow a path that leads to the

food source, which is considered their personal best solution (pbest). Each particle in the PSO algorithm searches for the best solution by considering its own past experiences and those of others in the group. The global best solution (gbest) is derived by observing particles that are closest to this optimal range. Each particle has a velocity for its pbest and an acceleration towards gbest. The primary objective of PSO is to converge towards the global optimal solution by moving each particle in the direction of pbest and gbest, with adjustable weights at each step. This algorithm is known for its enhanced exploration and exploitation capabilities [47, 48].

Since PSO typically operates in continuous space, it must be adapted for discrete problems such as IDS. In this adaptation, the particles' initial positions and velocities are defined in continuous space. Subsequently, a threshold is set, and binary conversion is applied. The fitness function is calculated for the particles in their binary form. The algorithm then computes the current velocities and positions of the particles in continuous space, applying the binary transformation process. This adaptation allows the PSO algorithm to be successfully used for discrete problems like IDS. The flow diagram for the PSO algorithm is illustrated in Fig. 3.

The flow diagram of the PSO algorithm used in this model is shown in Fig. 3. Initially, a population of 10 particles is generated, with each particle representing a set of features in the dataset. The positions and velocities of the particles for all features are initialized randomly. These positions are then binary transformed, where a threshold value of 0.5 is applied: any value less than the threshold is set to 0, while values greater than the threshold are set to 1. Next, the fitness function, as described in Eq. 1, is calculated based on the feature selection dataset. The error rate is determined by subtracting the classification accuracy obtained using the k-nearest (KNN) algorithm from 1. This error rate is assigned a weight of 90% in the fitness function. Additionally, the ratio of the number of selected features to the total number of features is given a weight of 10%. The resulting fitness function is computed for each particle. Each particle's personal best value (pbest) is then compared with its current fitness function value. If the current value (p) is better than the previous pbest, the new p value is assigned as the updated pbest. Afterward, the pbest values of all particles are compared, and the best pbest value is assigned to the global best (gbest) value. The velocity and position of each particle are updated according to the provided in Eqs. 1 and 2. This process is

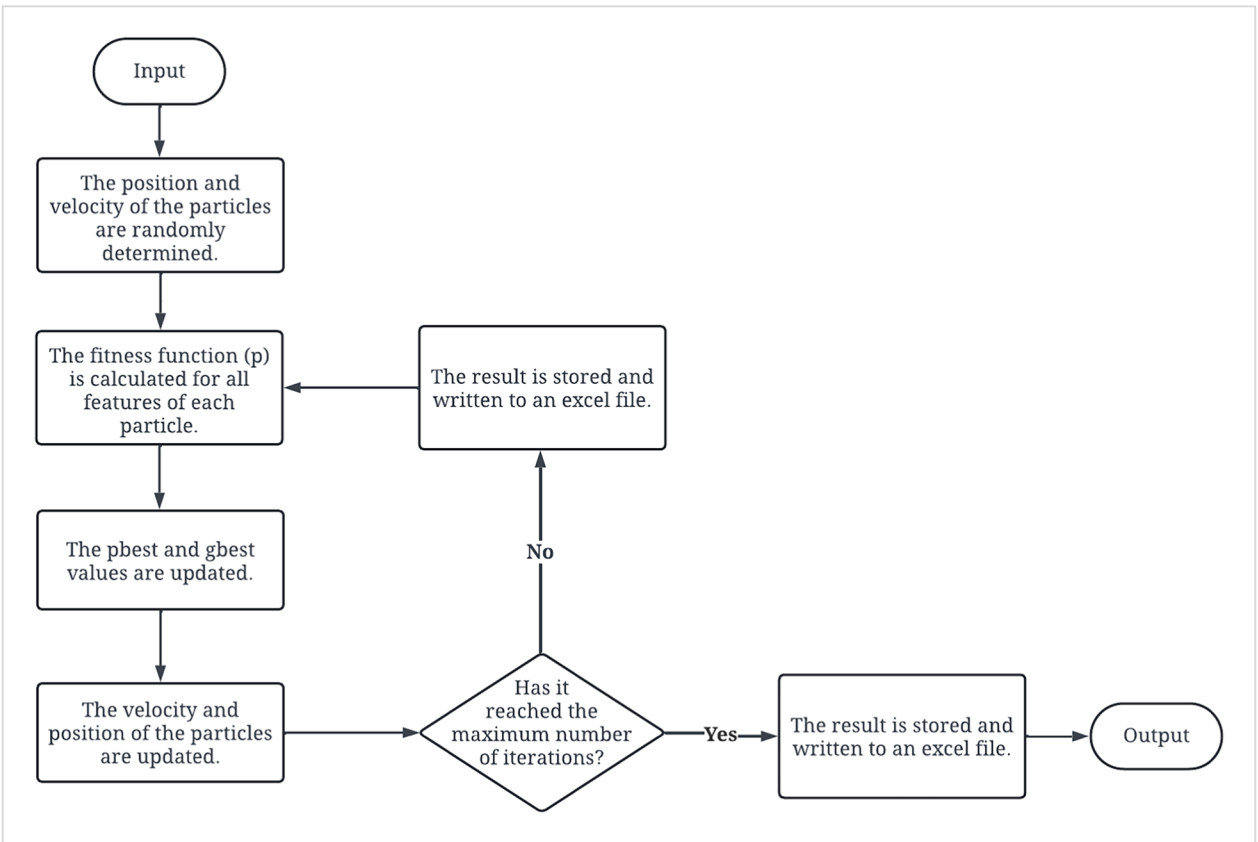


Fig. 3 PSO – flowchart

repeated iteratively until the maximum number of iterations is reached. Throughout the 200 iterations, the best gbest value is recorded in a log file. After the iterations are complete, the best gbest value is selected, and the corresponding features are chosen. The selected features, along with the total number of features, are also written in the log file.

$$p = \alpha * £ + \beta * \frac{SF}{TF} \tag{1}$$

$$v_{i+1} = v_i + c_1 * rand_1 * \left(pbest - x_i\right) + c_2 * rand_2 * \left(gbest - x_i\right) \tag{2}$$

$$x_{i+1} = x_i + v_{i+1} \tag{3}$$

In Eq. 1, p: fitness function, £: error rate, SF: number of selected features, TF : total number of features, α: 0.9 and β: 0.1. In Eqs. 2 and 3, x is the particle value, v is the rate of change of the particle, $c1$ and $c2$ are fixed values, *rand1* and *rand2* are random values.

Flower pollination algorithm Most plants are flowering plants, and there are over 250,000 species of flowering plants across the globe. Pollination serves as the primary reproductive strategy for these plants [49, 50]. It involves the transfer of pollen from one flower to another, facilitated either by wind or pollinators such as insects, birds, bats, and beetles. Flowering plants have evolved to produce nectar, attracting these pollinators and ensuring the process of pollination [51]. The mechanism of pollination in these plants can be categorized as follows:

Biotic (global) pollination: This form of pollination, also known as cross-pollination, is the primary method, accounting for nearly 90% of flowering plants. It involves pollinators like insects and birds, which move at varying speeds and can cover long distances, including flying. As such, pollen movement can span long distances, making this process akin to global pollination with potential levy flight characteristics [52–54]. In terms of optimization, this type of pollination can be viewed as a global search mechanism where pollen serves as the solution vector.

Abiotic (local) pollination: Self-pollination, or abiotic pollination, is a form of pollination that occurs without the need for a pollinator. It is estimated that about 10% of flowering plants rely on this method. Since this type of pollination occurs, often through wind or diffusion, the distance over which pollen is transferred is relatively short. Thus, this can be considered a form of local foraging [51, 54].

Flower stability: Flowering plants and pollinators, such as hummingbirds, sometimes form stable partnerships to conserve and/or increase the likelihood of success. This leads to the valuation of floral constancy, where pollinators specialize in visiting a specific set of flower species, avoiding the valuable cost of exploring new species. In return, flowering plants provide a steady nectar reward, encouraging frequent visits and maximizing their reproductive success [50, 51].

The features described above were used to develop an optimization algorithm known as the Flower Pollination Algorithm (FPA). Xin-She Yang introduced FPA in 2012, drawing inspiration from the fundamental characteristics of pollination [54]. The algorithm is based on four idealized rules, which are summarized as follows:

- Rule 1: Global pollination involves biotic and cross-pollination, where pollinators transfer pollen following Levy flights. This type of pollination allows for large-scale and long-distance movement, similar to a global search mechanism.
- Rule 2: Local pollination refers to abiotic and self-pollination processes that occur over shorter distances, typically involving local movements such as wind or diffusion.
- Rule 3: Flower constancy can be modeled as the probability of reproduction, which is proportional to the evaluation between two flowers. This concept reflects the consistency in pollinators, where pollinators are more likely to visit similar types of flowers.
- Rule 4: Transition probability (p∈[0,1]) governs the balance between local and global pollination. The transition probability can be influenced by external factors, such as wind, which alters the balance between local and global pollination. In many cases, local pollination tends to dominate the overall pollination dynamics.

As mentioned earlier, pollinators like birds and bats can transport pollen over long distances during biotic pollination, contributing to diversity and ensuring optimal pollination for reproduction. Hence, the first and third rules of the FPA can be mathematically formulated as follows in Eq. 4:

$$x_i^{t+1} = x_i^t + L(x_i^t - gbest) \tag{4}$$

In this context, x_i^t represents the pollen or solution vector at the current iteration, while gbest refers to the best solution identified among all solutions in that iteration. The parameter L denotes the strength of pollination, which corresponds to the step size. Levy flight effectively models this behavior, as pollinators travel long distances with unpredictable intervals [54]. The value of L is derived from the Levy distribution, as described in Eq. 5:

$$L \sim \frac{\lambda\Gamma(\lambda)sin(\pi\lambda/2)}{\pi}\frac{1}{s^{1+\lambda}}, s \gg s_0 > 0 \tag{5}$$

In Eq. 5, $\Gamma(\lambda)$ represents the standard gamma function, and this distribution is valid for steps where $s > 0$. It is generally advised to use $\lambda = 1.5$ [54]. As local pollination occurs through wind or diffusion without the involvement of any pollinator, both local pollination and floral constancy can be modeled by the equation provided in Eq. 6:

$$x_i^{t+1} = x_i^t + \varepsilon(x_j^t - x_k^t) \tag{6}$$

In Eq. 6, x_j^t and x_k^t represent pollen from different flowers within the same plant species. This equation simulates floral constancy within a limited neighborhood. If x_j^t and x_k^t belong to the same species within the same population, and we draw from a uniform distribution in the range [0, 1], the equation then represents a local random walk. The new solution vector generated by this process will not deviate significantly from the current solutions. From this, we can infer that both biotic and abiotic pollination are represented, though the proportion and frequency of each pollination type are not directly factored in. To account for this, a transition probability rule is implemented, where the value of P determines whether a solution update is influenced by local or global pollination. Although a simple value of $P = 0.5$ can be used, a more effective value of $P = 0.8$ typically yields better performance in most scenarios [54]. The FPA algorithm, which is a continuous optimization method based on plant pollination, operates in continuous space. However, since IDS are discrete problems, the FPA algorithm must be modified. To adapt, the initial population generated in continuous space is assigned a threshold, and binary conversion is performed. The fitness function is calculated for each feature after binary conversion. Then, local and global pollination is carried out in the continuous space. Afterward, a binary transformation is applied to the results, and the best solution is updated. This approach ensures that, while the FPA algorithm is originally designed for continuous space, it can be adapted for discrete problems. The flow diagram of the model is shown in Fig. 4.

In the initialization step of the algorithm outlined in Fig. 4, a population of 10 flowers is created, and the number of features in the dataset is determined. The initial positions and velocities for all features of each flower are randomized and then subjected to binary transformation. The binary transformation is carried out using a threshold value of (0.5) any value below this threshold is set to 0, while any value above it is set to 1. The fitness function, as described in Eq. 1, is then calculated for the feature selection dataset. The error rate is determined by subtracting the accuracy obtained from classifying the dataset using the k-nearest algorithm from 1. This error rate is given a weight of 90% in the fitness function. The ratio of selected features to the total number of features is weighted at 10%. In the first iteration, the best result (gbest) is updated by comparing the fitness values. Then, the transition probability value P for each feature is compared with a randomly generated value. If "rand < P", global pollination is applied via Levy flight. If "rand > P", local pollination takes place. After pollination, the fitness function is recalculated for the newly obtained features, and the gbest value is updated. This process is repeated until the maximum number of iterations is reached. The gbest value from each iteration is logged, and after 200 iterations, the best gbest value is selected. Finally, the selected features and their count are recorded in the log file.

Differential Evolution Differential Evolution (DE) is a well-regarded evolutionary algorithm introduced by Storn and Price as a robust optimization technique [55]. Like other evolutionary algorithms (EAs), DE uses key strategies such as mutation, crossover, and selection, applying them iteratively to converge on a globally optimal solution. Renowned for its efficiency, DE is considered one of the most effective evolutionary algorithms. In DE algorithm, each individual within the population is referred to as a target vector. Mutation is performed by perturbing this target vector using different vectors from other individuals in the population, generating a mutant vector. Subsequently, the crossover operation blends the parameters of the mutant vector with those of a parent vector, creating a trial vector. Finally, the selection process compares the trial vectors with their corresponding parent vectors and selects the vectors with the best fitness values to form the next generation. The core principle of DE, as a heuristic optimization approach, lies in leveraging information exchange among the population and the unique memory of each individual to progressively move closer to the global optimum. Through repeated mutation, crossover, and selection, the population evolves towards the optimal solution. As DE operates in continuous space, adapting it for IDS, a discrete problem, requires specific modifications. First, the initial population generated in continuous space undergoes a binary transformation based on a threshold value. After this transformation, the fitness function is evaluated for all genes of the binary-converted individuals. Mutation and crossover are then performed in continuous space, and the resulting vectors are again subjected to binary transformation. The best solution is updated based on the fitness evaluations. This approach allows DE, inherently a continuous optimization algorithm, to be effectively utilized for discrete problems. The flow diagram of the adapted DE algorithm proposed in this study is presented in Fig. 5.

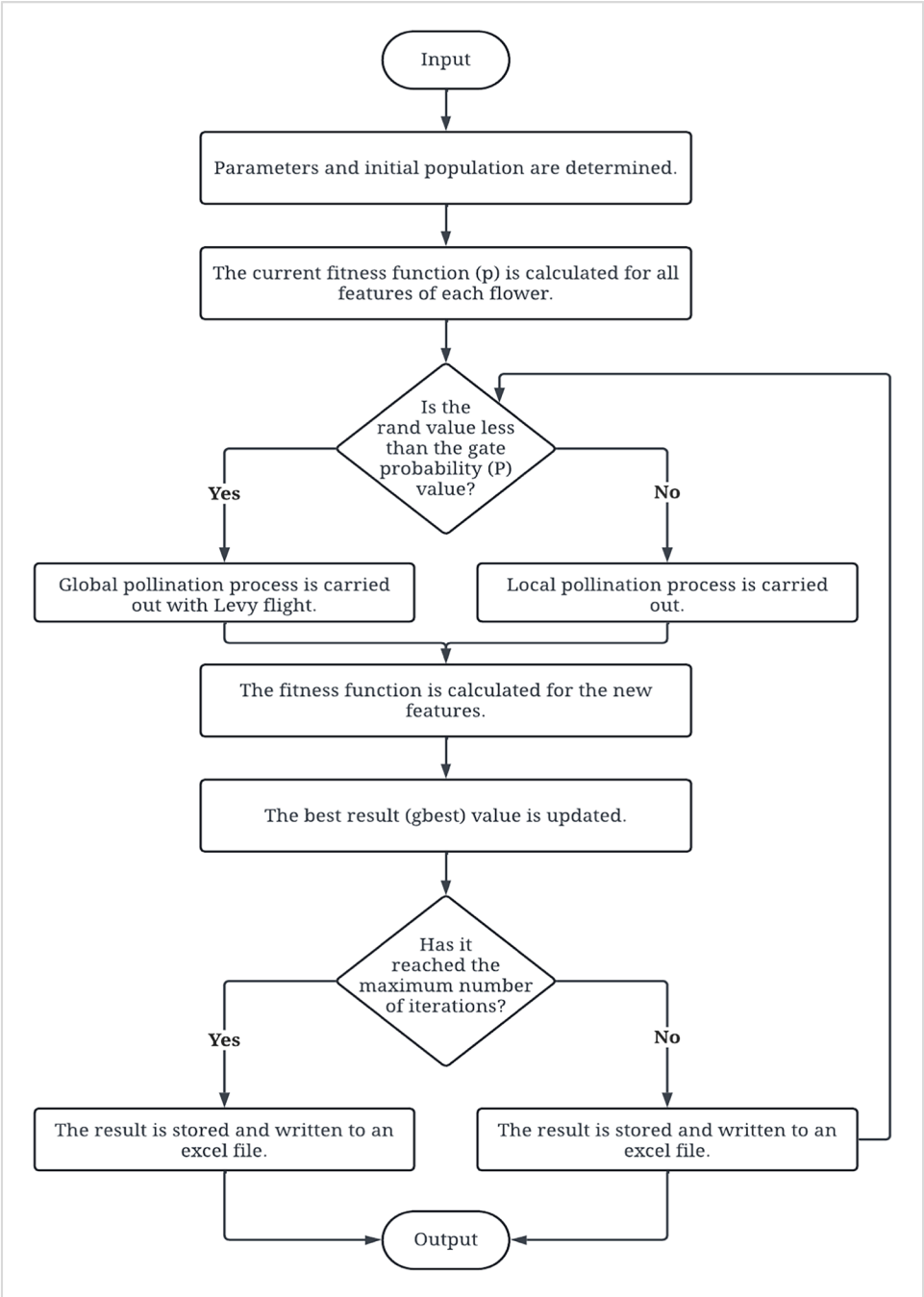


Fig. 4 Flowchart applied for FPA

Here, based on the studies in the literature, a chromosome population is initially created with the size of 10 individuals and the number of features in the dataset.

$$X_G = \{x_{1,G}, x_{2,G}, x_{3,G}, \cdots, x_{i,G}\} \tag{7}$$

Each chromosome $x_{i,G}$ in Eq. 7 is called the target vector. The starting position for all features of each individual is randomized. These are binary transformed. The binary transformation is written in such a way that a threshold value (0,5) is set such that 0 is less than the threshold and 1 is greater than the threshold. After this process, the fitness function given in Eq. 1 is calculated over the feature selection dataset. The error rate is obtained by subtracting the accuracy rate obtained after the classification of this data set using the k nearest neighbor algorithm from 1. The weight of this to the fitness function is set to 90%.

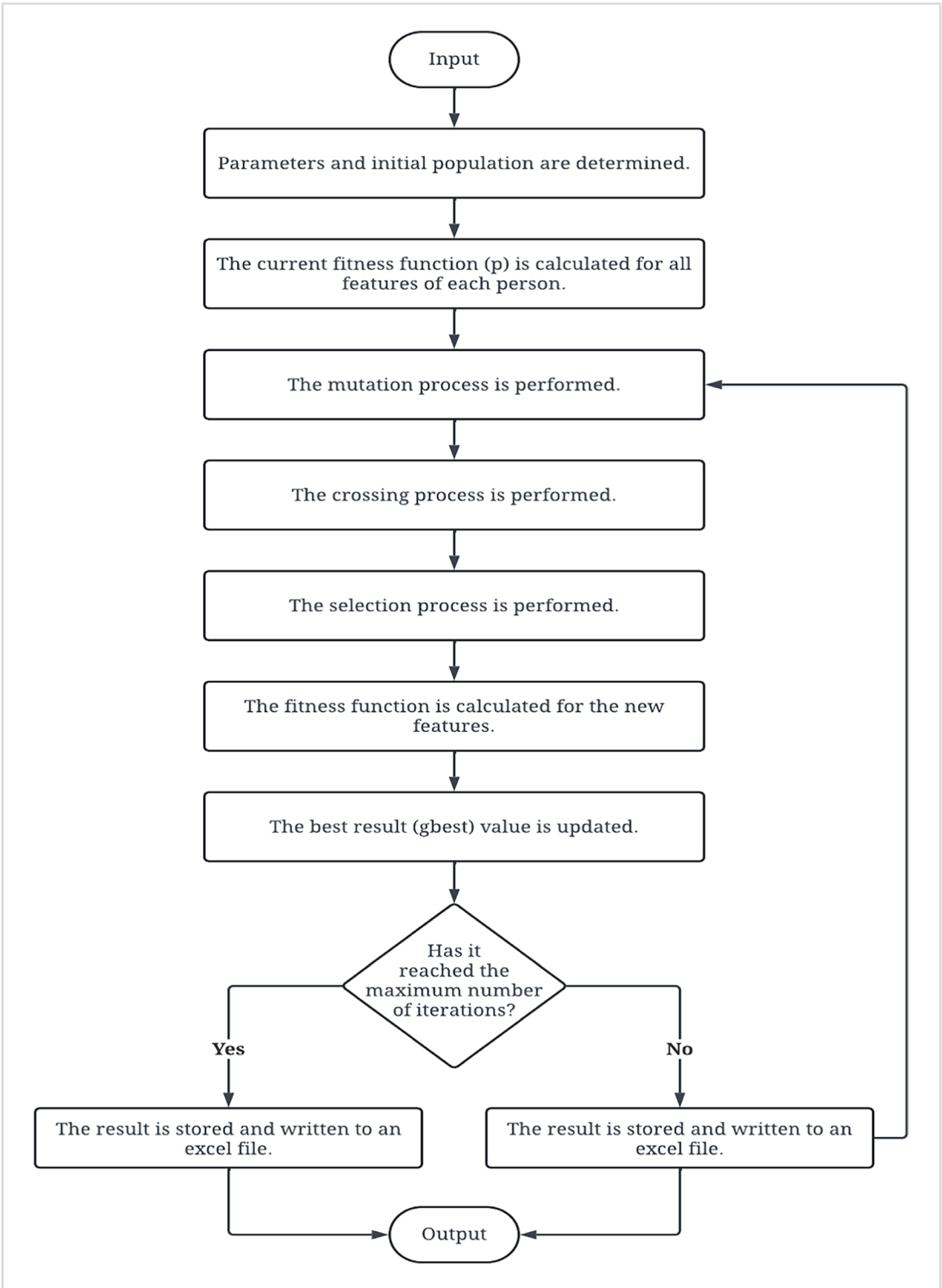


Fig. 5 Flowchart for feature selection using DE

The weight of the ratio of the number of selected features to the total number of features was set as 10% and the fitness function was calculated. For the first iteration, the best result value (gbest) is updated by comparing the fitness function value. Then the mutation process is started, where for each $x_{i,G}$ a donor vector $v_{i,G+1}$ is created as given in Eq. 8.

$$v_{i,G+1} = x_{r1,G} + F.(x_{r2,G} - x_{r3,G}) \tag{8}$$

In Eq. 8, $x_{r1,G}$, $x_{r2,G}$, $x_{r3,G}$ are randomly selected from the population except the target vector. F is a user-defined mutation or constant factor (F = 0.5) that controls the amplification of the differential variation ($x_{r2,G} - x_{r3,G}$). The crossover process is then started. The donor vector, i.e., the mutant vector, is mixed with the target vector to produce a trial vector $u_{i,G+1}$ given in Eq. 9:

$$u_{i,G+1} = \begin{cases} v_{j,i,G+1} \ if \ rand_{j,i} \leq CR \ or \ j = I_{rand} \\ x_{j,i,G} \ if \ rand_{j,i} > CR \ and \ j \neq I_{rand} \end{cases} \tag{9}$$

In Eq. 9, $i = 1,2,\ldots,N$; $j = 1,2,\ldots,D$ denotes the serine, where N is the population size and D is the size ofx_i. CR is the crossover constant (CR = 0.9). I_{rand} ensures that$v_{i,G+1} \neq x_{i,G}$. After the crossover process is complete, the selection process is started. The trial vector $u_{i,G+1}$ in Eq. 10 and the target vector $x_{i,G}$ are evaluated using the fitness function. A comparison is made between the two and the one with the lower value is passed on to the next generation.

$$x_{i,G+1} = \begin{cases} u_{i,G+1} & if \ f\ (u_{i,G+1}) \leq f(x_{i,G}) \\ x_{iG} & other \end{cases} \tag{10}$$

For the new features obtained, the fitness function is calculated and the gbest value is updated. These steps are repeated until the maximum number of iterations is reached and the gbest value of each iteration is written to the log file. Among 200 iterations, the best gbest value is taken into account for feature selection. The selected features and their count are recorded in the log file.

4.2.2 Classification algorithms

In this section, we will focus on the machine learning algorithms we used to emphasize the purpose and success of this research. Machine learning is a field of study that enables computers to automatically learn from large data sets by applying mathematical models [56]. Machine learning algorithms uncover patterns, make predictions, and solve complex problems using data-driven approaches without being explicitly programmed for each task. In this section, machine learning algorithms used for intrusion detection systems are described. These algorithms aim to analyze network traffic and identify normal and malicious behavior (*abnormal*).

"Logistic Regression (LR)" is a machine learning classification algorithm employed to analyze datasets with one or more independent variables and categorical dependent variables [57]. Unlike linear regression, which predicts continuous numeric outcomes, logistic regression applies the logistic sigmoid function to its output, converting it into a probability. This probability is then mapped to one or more discrete classes [58].

Several DT models are discussed in the literature, including C4.5, CART, and ID3. In this study, the C4.5 algorithm is employed for classification. C4.5 is a supervised machine learning technique that constructs decisions and potential outcomes in a tree-like structure. It divides the data space into smaller sub-domains, assigning each partition a class label. The decision tree begins with a root node, and at each internal node, the data is split based on the most informative feature, with the leaf nodes representing the final class outcomes [59, 60].

"Random forest (RF)" operates as an ensemble of multiple decision trees. The class prediction is made based on the majority vote from each tree, with the most frequent class outcome being selected [61]. RF integrates the outputs of various trained classifiers within the ensemble to classify new instances. Each tree utilizes independent random vectors and casts a vote for the most popular class [62]. RF is a powerful classifier, particularly effective on large datasets with up to 400 features, and is favored in machine learning for its ability to handle imbalanced or missing data. Additionally, the use of multiple trees enhances robustness against outliers, thereby improving the reliability of the algorithm [63].

"K-nearest neighbor (KNN)" is a nonparametric, supervised learning algorithm that determines the likelihood of a data point belonging to a specific class by comparing it with other points in its vicinity. KNN is widely utilized due to its simplicity, consistency, and effectiveness in various applications, such as intrusion detection [64]. In both classification and regression tasks, KNN classifies a new data point based on the classes or values of its k-nearest neighbors from the training set. Known as a lazy learning algorithm, KNN does not perform any calculations when the training data is provided,it only builds the model when a query is made [62]. The performance of KNN heavily relies on the selection of k, the distance metric, and the decision rules used (G. [65]).

"Naive Bayes (NB)" algorithm is a commonly used probabilistic machine learning technique for classification tasks.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \tag{11}$$

Bayes' theorem, as shown in Eq. 11, expresses "the probability that event A will occur given that event B has occurred" [66]. In this context, A represents the hypothesis and B represents the evidence. The Naive Bayes algorithm is particularly useful for intrusion detection in binary scenarios, where network traffic is categorized as either normal or an attack. In a study involving the KDD CUP 99 dataset, an accuracy rate of 85% was achieved [67]. For datasets containing both categorical and numerical data, categorical data is often transformed into numerical data through methods like

one-hot or label encoding as part of the preprocessing step, enabling more effective classification with Naive Bayes.

The boosting technique focuses on constructing and combining multiple weak models to enhance performance. Both "gradient boosting" and "extra gradient boosting" are based on the gradient boosting (GB) principle, which strengthens the model's capabilities. These classifiers, like other techniques in machine learning, are supervised learning algorithms. Modern applications typically implement them on multicore computers using Python, though they can also be executed in other programming languages. These algorithms significantly improve classification accuracy, particularly when applied to training and test datasets [68]. Overall, gradient boosting offers key features such as parallelization, regularization, non-linearity, cross-validation, and scalability.

"Linear Discriminant Analysis (LDA)" is a data analysis technique aimed at identifying a lower-dimensional subspace and comparing it to the original sample space. The primary objective is to establish a linear relationship among categorical data features that can effectively distinguish them from one another [69]. LDA plays a crucial role in computing systems' network and port attack detection. It is a valuable machine-learning algorithm for intrusion detection [70]. One of the key advantages of LDA is its ability to apply normal distribution assumptions for inference, feature selection, and prediction. However, its effectiveness may be limited when working with categorical variables.

"Quadratic Discriminant Analysis (QDA)" shares similar characteristics with Linear Discriminant Analysis (LDA), but it allows for different covariance matrices for each class. The QDA classifier is based on Bayesian discriminant analysis and is widely used in various domains, such as healthcare, where it helps analyze blood cell images to differentiate between healthy and unhealthy samples. In machine learning, QDA is applied to tasks like identifying port scanning and DoS attacks [71].

"AdaBoost (AB)" is a well-known boosting algorithm that constructs ensemble models by combining multiple weak classifiers. Its key strength lies in the ability to aggregate many weak learners, typically decision trees that make simple decisions, to create a robust classifier. In AdaBoost, each weak learner is assigned a weight, influencing its impact on the overall model. The ensemble's prediction is made through "soft voting," where the weighted votes of individual learners are considered. However, in certain situations where the predictions deviate from expectations, the weights of weaker learners are redistributed equally, leading to "hard voting" [72].

An artificial neural network (NN), is a machine learning model inspired by the human brain's learning process, first introduced in 1943 based on the structure of brain cells [73]. Deep learning, a subfield of machine learning, leverages neural networks to solve complex problems and is widely used in sectors like IoT, industry, and healthcare [74]. Neural networks are capable of detecting various types of attacks due to their ability to monitor networks and predict potential intrusions. However, limitations in the training process can reduce their accuracy [75], which is why they are often combined with other models to enhance performance. Neural networks consist of artificial neurons arranged in layers, typically including input, hidden, and output layers. Data is processed through the hidden layers, and the final result is produced in the output layer. Connection weights govern the interaction between neurons and allow the network to learn more detailed information as data flows through each layer. Deep learning is effective for handling complex, high-dimensional data and provides generalization capabilities [76]. These algorithms excel at training non-linear models on large datasets, making them particularly useful for network intrusion detection. Modern deep learning techniques require thousands of labeled examples and need millions of examples to outperform human performance. While earlier datasets were smaller, more recent datasets such as ISCX IDS 2012, CIC IDS 2017, and CIC [42] provide ample data to evaluate deep learning models.

5 Experimental results

This section presents the results and evaluations of the experimental studies. In the proposed model, the data sets are first subjected to classification by algorithms without feature selection and analyzed using evaluation criteria. The classification process here is referred to as SS. In the second stage, PSO, FPA, and DE methods used for feature selection were applied to the datasets separately. Finally, the selected features were evaluated by the classifiers using the performance criteria specific to the dataset, and the outputs were analyzed. The running times of the algorithms were also compared. The results and evaluations for each dataset are described in the following subsections.

5.1 Studies on KDD CUP 99

The studies conducted on the KDD CUP 99 dataset were analyzed using time, accuracy, precision, recall, and F1 score metrics, which are presented in the following tables and figures. Additionally, the gbest values of the PSO, FPA, and DE algorithms for 200 iterations are illustrated in Fig. 6.

As seen in Fig. 6, the best gbest value was obtained for FPA, PSO, and DE after approximately 180th iteration. The analysis results for duration are given in Fig. 7.

Figure 7 demonstrates that, according to the SS results, the application of feature selection approaches led to a

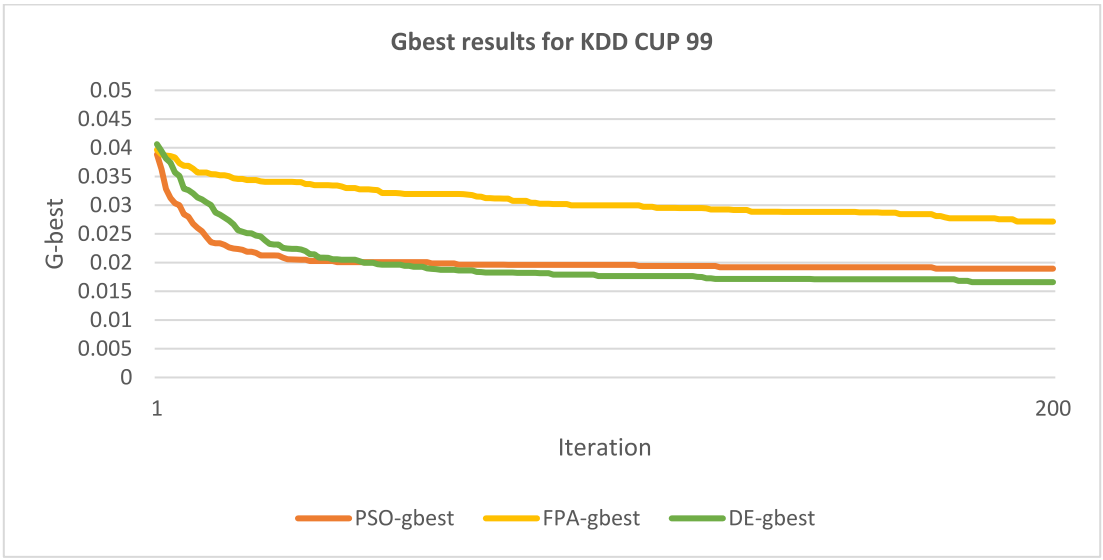


Fig. 6 KDD CUP 99: iteration—Gbest results

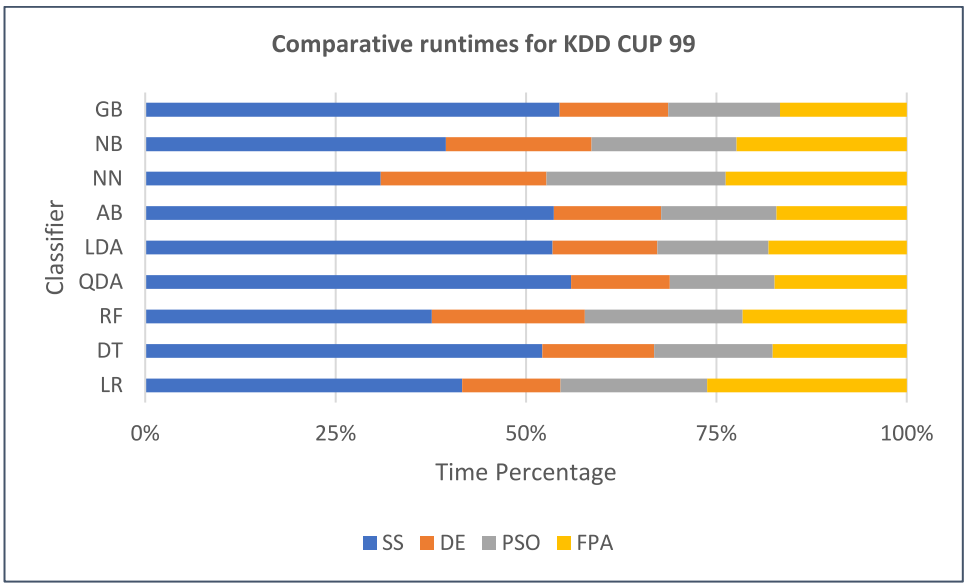


Fig. 7 KDD CUP 99: classifier—time results

significant reduction in classification times, decreasing them to one-third of the original times. This improvement in time performance suggests that feature selection not only helps streamline the classification process but also enhances computational efficiency. Additionally, the overall performance in terms of time was notably improved, allowing for faster processing without sacrificing accuracy. For a detailed evaluation of the model's effectiveness, the results of the analysis based on the key performance metrics—accuracy, precision, recall, and F1-score—are provided in Table 1, offering a comprehensive comparison of performance across various criteria. The evaluation metrics provide a comprehensive evaluation of the model's ability to classify network traffic accurately, reflecting both the correctness (accuracy), the proportion of true positives (precision), the model's ability to detect actual positive instances (recall), and the balance between precision and recall (F1 score). The values for each of these measures are presented in the table to highlight the effectiveness of the models used in the context of intrusion detection.

Table 1 KDDCUP99—Results of classifiers

Dataset KDDCUP99 and classification results										
		LR	DT	RF	QDA	LDA	AB	NN	NB	GB
Accuracy	SS	**0,9935**	0,9800	**0,9969**	0,9636	0,9432	**0,9899**	**0,9899**	0,9401	0,9868
	DE	0,9834	**0,9855**	0,9948	**0,9642**	**0,9714**	0,9892	0,9856	**0,9563**	**0,9955**
	PSO	0,9752	**0,9855**	0,9966	**0,9719**	**0,9806**	0,9872	0,9856	0,9076	**0,9950**
	FPA	0,9862	**0,9841**	0,9946	0,9596	**0,9809**	0,9855	0,9880	**0,9798**	**0,9917**
Precision	SS	**0,9980**	0,9977	**0,9999**	**0,9997**	0,9961	**0,9970**	**0,9990**	**0,9926**	0,9955
	DE	0,9958	**0,9990**	0,9997	0,9987	0,9956	0,9964	0,9977	0,9612	**0,9985**
	PSO	0,9910	**0,9986**	0,9997	0,9987	0,9939	0,9955	0,9984	0,9820	**0,9990**
	FPA	0,9966	**0,9995**	0,9996	0,9991	**0,9969**	0,9960	0,9966	0,9922	**0,9987**
Recall	SS	0,9940	0,9774	0,9963	0,9549	0,9326	0,9905	0,9884	0,9316	0,9880
	DE	0,9901	**0,9869**	0,9961	**0,9826**	**0,9964**	**0,9920**	0,9854	**0,9937**	**0,9961**
	PSO	**0,9943**	**0,9932**	**0,9965**	**0,9898**	**0,9961**	**0,9912**	**0,9930**	**0,9933**	**0,9974**
	FPA	0,9882	**0,9963**	0,9957	0,9524	**0,9813**	0,9902	**0,9886**	**0,9826**	**0,9925**
F1 Score	SS	**0,9959**	0,9872	**0,9981**	0,9759	0,9605	**0,9936**	**0,9935**	0,9585	0,9917
	DE	0,9894	**0,9907**	0,9967	**0,9765**	**0,9820**	0,9932	0,9908	**0,9732**	**0,9972**
	PSO	0,9845	**0,9910**	0,9979	**0,9819**	**0,9877**	0,9918	0,9908	**0,9831**	**0,9969**
	FPA	0,9912	**0,9902**	0,9966	0,9732	**0,9878**	0,9909	0,9924	**0,9872**	**0,9948**

The outputs in Table 1 are evaluated according to different feature selection methods. When the classifiers using feature selection with SS are compared, it is seen that the DE method has higher accuracy in DT, QDA, LDA, NB, and GB; PSO method has higher accuracy in DT, RF, QDA, LDA, and GB; and FPA method has higher accuracy in DT, LDA, NB, and GB. In terms of the precision criterion, when the classification results using SS and feature selection approaches are compared, it is seen that DE, PSO, and FPA methods obtain better results in DT and GB. Also, the results by the LDA classifier provided better results in FPA compared to other feature selection methods. Regarding the recall metric, the comparison of classification results between SS and feature selection approaches reveals that the DE, PSO, and FPA methods achieve superior performance in DT, QDA, NB, and GB. PSO method has performed as the most effective feature selector across all classifiers, delivering the best results and achieving superior outcomes in LR and RF compared to the others. Based on the F1-Score metric, the comparison of classification results between SS and feature selection approaches indicates that the DE, PSO, and FPA methods yield superior performance in DT, LDA, NB, and GB. In the outputs with the LR, RF, AB, and NN classifiers, it is understood that the feature selection approaches are not successful at the expected level. In contrast, other algorithms utilizing feature selection approaches have been observed to achieve more successful outcomes in a shorter time.

5.2 Studies on NSL-KDD

The studies for the NSL-KDD dataset were analyzed using time, accuracy, precision, recall, and F1 score metrics. These are given below in tables and figures, respectively. The gbest values of PSO, FPA and DE algorithms for 200 iterations are given in Fig. 8 below.

As seen in Fig. 8, the best gbest value was obtained for FPA, PSO and DE after approximately 170th iteration. The results of the analysis for duration are given in Fig. 9. When Fig. 9 is analyzed, it is seen that the performance in terms of time is improved by reducing the classification times to 1 in 3 when feature selection approaches are used according to the SS results.

The results of the analysis of the Accuracy, Precision, Recall, and F1 Score performance measures for the tests on the NSL-KDD dataset are given in Table 2.

According to the outputs in Table 2, when the SS classifiers and feature selection are compared in terms of the accuracy metric, DE, PSO, and FPA methods achieve better results in DT. Respectively, the best results were obtained with FPA in the RF classifier, PSO in the QDA classifier, and DE in the GB classifier. This indicates that the outputs obtained by using feature selection methods across different classifiers do not demonstrate consistent performance. Therefore, the goal should be to test different feature selection methods across all possible classifiers to achieve the best performance. In terms of the precision criterion, when the classifiers with SS and feature

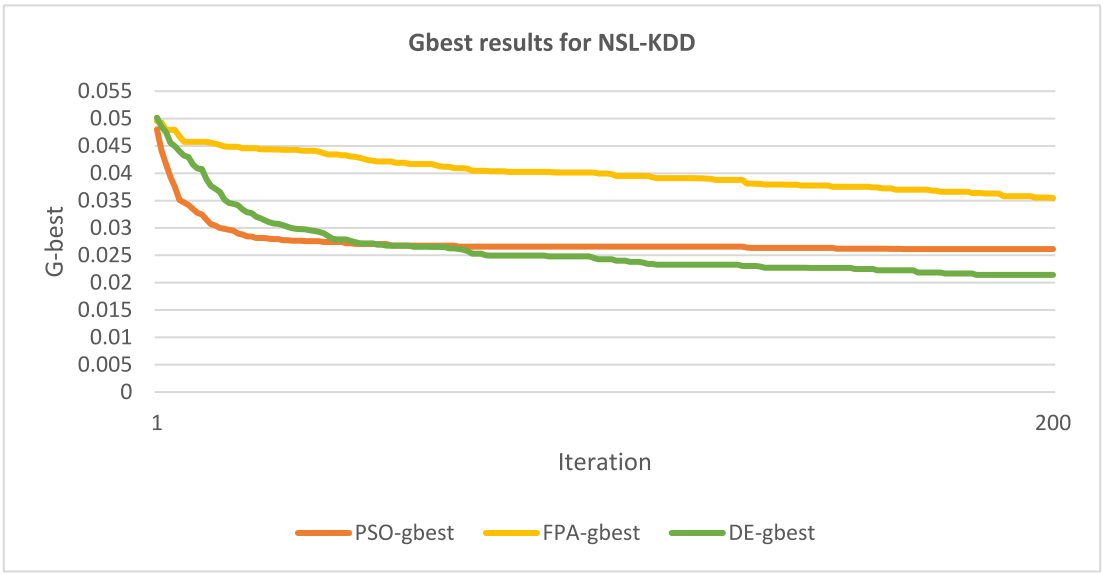


Fig. 8 NSL-KDD: iteration—Gbest results

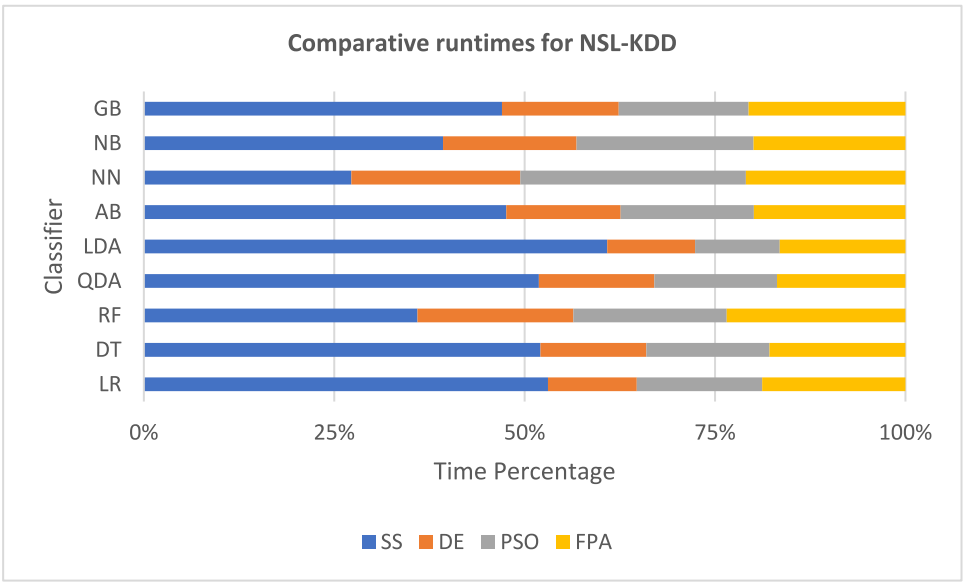


Fig. 9 NSL-KDD: classifier—time results

selection are compared, DE, PSO, and FPA algorithms achieve better results in DT, QDA, and NB. According to this metric, all classifiers using feature selection approaches, except for AB and MN classifiers, have achieved more successful results. The RF and LDA classifiers, using the FPA method, achieved the best performance; overall, the results indicate that the FPA method is the most successful feature selection approach. Regarding the recall metric, when the classifiers with SS and feature selection are compared, it is seen that DE, PSO, and FPA methods achieve better results in DT, QDA, AB, and GB. In terms of the F1-Score criterion, when the classifiers with SS and feature selection are compared, it is seen that DE, PSO, and FPA methods achieve better results in DT. According to these results, it is understood that the DT classifier obtains more successful outputs by using feature selection. Additionally, LR, RF, QDA, NB, and GB algorithms have achieved the best results with at least one feature selection method.

Table 2 Results of classifiers for NSL-KDD

Dataset NSL-KDD and classification results										
		LR	DT	RF	QDA	LDA	AB	NN	NB	GB
Accuracy	SS	0,**9460**	0,8870	0,9864	0,8369	**0,8933**	**0,9417**	**0,9599**	**0,8363**	0,9641
	DE	0,8499	**0,8912**	0,9861	0,8280	0,8461	0,9383	0,9257	0,7952	**0,9671**
	PSO	0,8782	**0,8958**	0,9813	**0,8522**	0,8576	0,9322	0,9176	0,8272	0,9630
	FPA	0,9143	**0,9108**	**0,9879**	0,8287	0,8522	0,9408	0,9294	0,8092	0,9627
Precision	SS	0,9385	0,9132	0,9886	0,9695	0,9518	**0,9499**	**0,9503**	0,9673	0,9607
	DE	**0,9453**	**0,9688**	0,9860	**0,9908**	0,9442	0,9428	0,9275	**0,9708**	**0,9640**
	PSO	0,9380	**0,9318**	0,9866	**0,9853**	0,9389	0,9443	0,9335	**0,9832**	**0,9614**
	FPA	**0,9480**	**0,9726**	**0,9895**	**0,9844**	**0,9554**	0,9419	0,9462	**0,9842**	0,9604
Recall	SS	**0,9686**	0,8857	0,9875	0,7376	**0,8559**	0,9475	**0,9808**	0,7373	0,9770
	DE	0,8298	**0,9497**	**0,9896**	**0,7907**	0,7969	**0,9509**	0,9391	**0,7738**	**0,9825**
	PSO	0,8688	**0,9589**	0,9863	**0,9521**	0,8063	**0,9501**	0,9700	**0,9524**	**0,9814**
	FPA	0,9221	**0,9903**	**0,9900**	**0,8039**	0,8127	**0,9553**	0,9621	0,7286	**0,9779**
F1 Score	SS	**0,9533**	0,8989	0,9880	0,8364	**0,9013**	**0,9487**	**0,9653**	0,8368	0,9687
	DE	0,8557	**0,9060**	0,9878	**0,8381**	0,8516	0,9461	0,9353	0,8061	**0,9713**
	PSO	0,8872	**0,9112**	0,9836	**0,8666**	0,8650	0,9406	0,9306	**0,8377**	0,9679
	FPA	0,9245	**0,9267**	**0,9894**	0,8333	0,8623	0,9484	0,9384	0,8072	0,9675

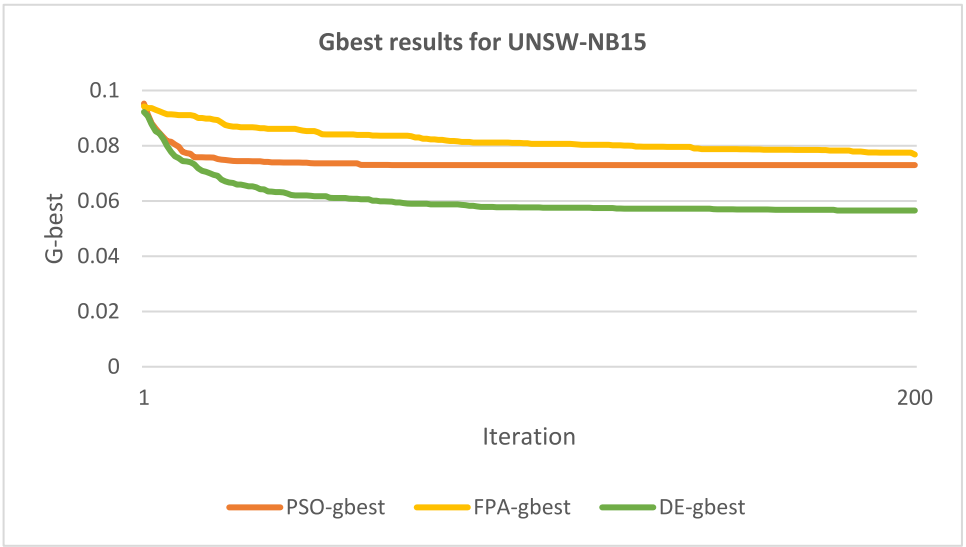


Fig. 10 UNSW-NB15: iteration—Gbest results

5.3 Studies on UNSW-NB15

The studies for the UNSW-NB15 dataset were analyzed using time, accuracy, precision, recall and F1-score metrics. These are given below in tables and figures respectively. The gbest values of PSO, FPA and DE algorithms for 200 iterations are given in Fig. 10 below.

As seen in Fig. 10, the best gbest value was obtained for FPA, PSO and DE after approximately 175th iteration. The analysis results for duration are shown in Fig. 11.

When analyzing Figs. 10 and 11, a notable improvement in classification times is evident across the different feature selection approaches. According to the SS results, the use of the DE algorithm resulted in a reduction of classification time to one-third of the original duration, demonstrating a substantial enhancement in computational efficiency. In comparison, the application of PSO and FPA algorithms led to an even more significant decrease, reducing the classification time to one-fourth of the initial time. This decrease in time highlights

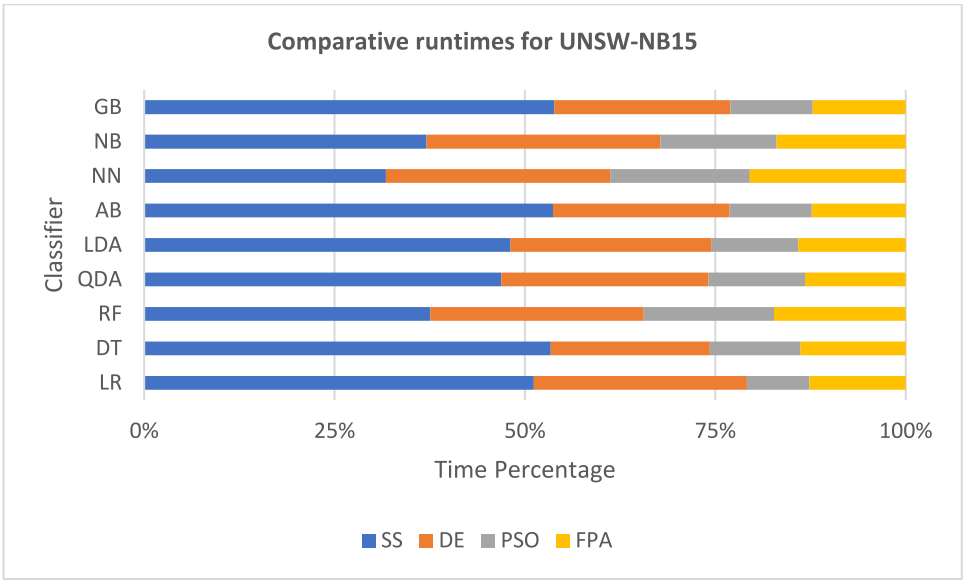


Fig. 11 UNSW-NB15: classifier—time results

Table 3 Results of classifiers for UNSW-NB15

Dataset UNSW-NB15 and classification results										
		LR	DT	RF	QDA	LDA	AB	NN	NB	GB
Accuracy	SS	0,8566	0,8633	**0,9545**	0,8082	0,8267	**0,9084**	0,8764	0,7706	**0,9268**
	DE	**0,9002**	**0,9244**	0,9098	**0,8618**	**0,8935**	0,9078	**0,9203**	**0,8419**	0,9073
	PSO	0,7346	**0,8968**	0,9471	0,7878	0,7369	0,8797	0,8531	0,7573	0,9198
	FPA	0,7932	**0,8671**	0,9486	0,7911	0,7566	0,8967	0,8691	0,7689	0,9240
Precision	SS	0,8833	0,9362	**0,9666**	0,7860	0,8388	**0,9281**	0,9103	0,8399	**0,9414**
	DE	**0,9042**	0,9130	0,9272	**0,9241**	**0,9174**	0,9186	**0,9163**	**0,9407**	0,9204
	PSO	0,8779	**0,9446**	0,9537	**0,8911**	0,7640	0,8993	0,8724	**0,8941**	0,9273
	FPA	0,8008	**0,9467**	0,9633	**0,8852**	0,7745	0,9202	0,8799	**0,9019**	0,9349
Recall	SS	0,8722	0,8103	0,9520	0,9826	0,8981	0,9100	0,8753	0,7259	0,9286
	DE	**0,9798**	**0,9998**	**0,9583**	**0,9916**	**0,9873**	**0,9871**	**0,9909**	**0,9828**	**0,9659**
	PSO	0,8515	**0,9005**	**0,9527**	**0,9862**	0,8560	**0,9256**	**0,9411**	**0,9281**	**0,9335**
	FPA	**0,9382**	**0,9115**	**0,9495**	**0,9974**	**0,9152**	**0,9239**	**0,9543**	**0,9907**	**0,9317**
F1 Score	SS	0,8700	0,8640	**0,9583**	0,8621	0,8566	0,9162	0,8822	0,7762	0,9333
	DE	**0,9351**	**0,9507**	0,9379	**0,9007**	**0,9310**	**0,9389**	**0,9476**	**0,8956**	**0,9372**
	PSO	0,7911	**0,9068**	0,9521	0,8086	0,7936	0,8975	0,8812	**0,7859**	0,9289
	FPA	0,8430	**0,8879**	0,9524	0,8483	0,8166	0,9094	**0,8924**	0,7647	0,9318

the effectiveness of these meta-heuristic-based feature selection methods in accelerating the classification process while maintaining or improving performance. The reduction in classification time is particularly valuable in real-time intrusion detection systems, where speed and efficiency are crucial. Furthermore, these results emphasize that not only do these feature selection methods reduce processing time, but they also contribute to the overall improvement of the classification model's efficiency, making them a powerful tool for optimizing both performance and computational resources. The results of the analysis in terms of Accuracy, Precision, Recall and F1-score performance criteria as classification outputs are given in Table 3. Thus, in addition to comparing the runtime performance of the algorithms, a comprehensive evaluation of their performance in terms of key metrics has also been conducted. This allows for a more thorough understanding of how each algorithm performs not only in terms of computational efficiency but also in terms of classification effectiveness. By assessing both

runtime and classification performance, a balanced view is provided to determine which algorithms offer the best trade-off between speed and accuracy, ensuring optimal performance for real-world applications.

Based on the accuracy results in Table 3, the comparison between the SS classifiers and those using feature selection methods reveals that DE, PSO, and FPA methods achieve better results in DT, while the DE algorithm outperforms others in LR, QDA, LDA, NN, and NB. In terms of the precision metric, when the SS classifiers and other methods are compared, it is seen that classifiers with DE, PSO, and FPA achieve better results in QDA and NB. When evaluated according to the recall metric in Table 3, the best results were obtained with DT, RF, QDA, AB, NN, NB, and GB classifiers using feature selection methods. These results demonstrate the strength of feature selection in identifying true positives. In this case, the PSO method was the only feature selection technique utilized, with the LR and LDA classifiers yielding relatively poor results. In contrast, all other results underscore the effectiveness of feature selection approaches on the dataset UNSW-NB15. When evaluating the results based on the F1-Score, all classifiers using feature selection approaches, except for the RF classification algorithm, achieved the best scores. In the classifier RF, the results with SS and feature selection methods were comparable. The results show that feature selection methods generally improve performance across most classifiers. Additionally, the DT classifier achieved the best performance when using all feature selection methods.

5.4 Studies on CSE-CIC-IDS2018

The studies for the CSE-CIC-IDS2018 dataset were analyzed using time, accuracy, precision, recall, and F1-score metrics. These are given below in tables and figures, respectively. The gbest values of PSO, FPA, and DE algorithms for 200 iterations are given in Fig. 12.

As shown in Fig. 12, the best gbest value was obtained for FPA, PSO, and DE after approximately 160th iteration.

Analysis results for duration are given in Fig. 13. When the graph is analyzed, it is seen that when feature selection approaches are used according to the SS results, the classification times are reduced up to 1 in 6, and the performance in terms of time is increased. These outputs are given in Fig. 13.

The results of the analysis in terms of Accuracy, Precision, Recall, and F1 Score criteria are presented in Table 4.

The evaluation of the test results for the CSE-CIC-IDS2018 dataset in Table 4 is based on feature selection methods. Since the results are very close, additional decimal places have been showed for greater precision. Regarding the accuracy criteria, applying feature selection approaches in DT, RF, NN, and GB enhanced performance, while the results for NB were generally unchanged. According to the precision metric, all the best results, except for LR and LDA, were achieved with classifiers that used feature selection approaches. Although the NB classifier achieved the best result with a value of 0.99614 for SS in the recall metric, the results of all other classifiers with at least one feature selection method were more successful. For example, all feature selection method results in RF, LDA, NN, and GB

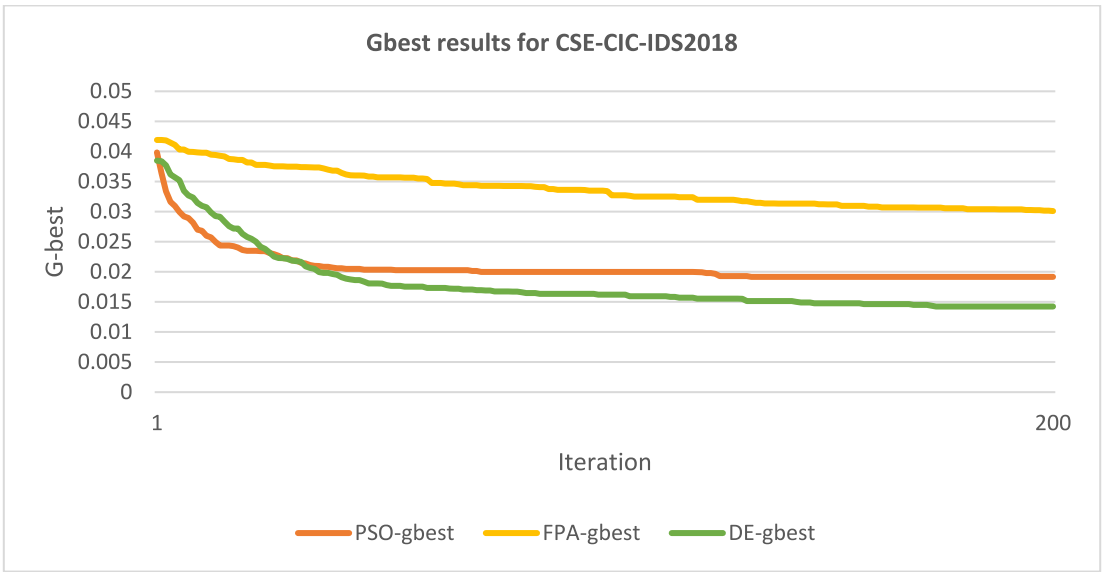


Fig. 12 CSE-CIC-IDS2018: iteration—Gbest results

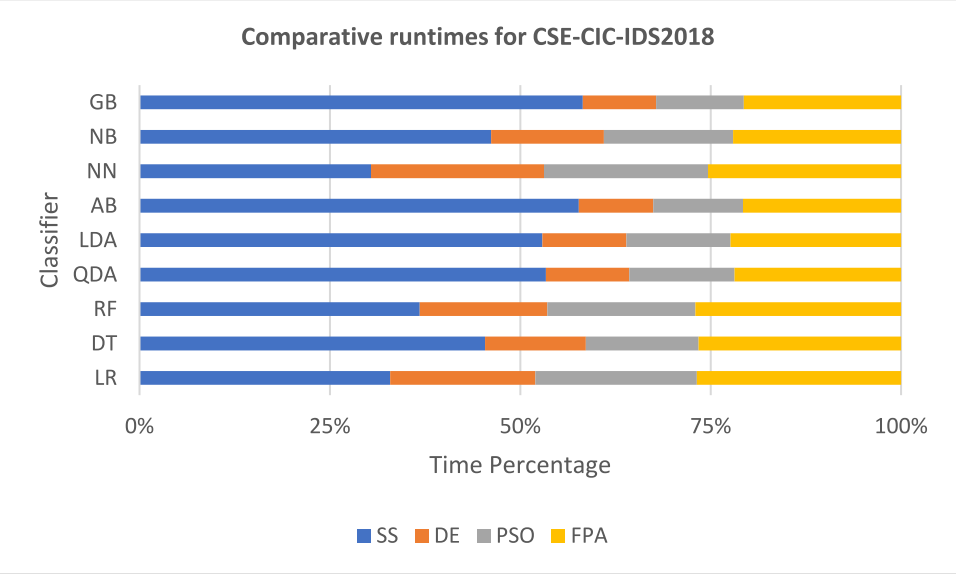


Fig. 13 CSE-CIC-IDS2018: classifier—time results

presented high performances. Lastly, when examining the results with the F1-score, the use of three different feature selection methods individually in DT, RF, and GB yielded the best results. Similar to the other metric results, these F1-score results also demonstrate that using feature selection generally improves performance and even enables achieving better results in a shorter time.

The results of the proposed study are presented in different subsections by evaluating them separately according to the data sets. Along with these results, a comparative analysis of the results of the studies in the literature and the results of the proposed model is presented in Table 5. While making these evaluations; among the studies in the literature, studies that use KDD CUP 99, NSL-KDD, UNSW-NB15, and CSE-CIC-IDS2018 and generally use heuristic algorithms for feature approximation are compared. The studies reviewed here for comparison purposes are generally studies that consider outputs based on time and/or performance metrics (accuracy-*acc*, precision-*pre*, recall-*rec*, and f1 score-*f1*). In addition, studies that are similar in concept

Table 4 Results of classifiers for CSE-CIC-IDS2018

Dataset CSE-CIC-IDS2018 and classification results										
		LR	DT	RF	QDA	LDA	AB	NN	NB	GB
Accuracy	SS	**0,99970**	0,99902	0,99957	0,99957	**0,99883**	**0,99998**	0,99876	**0,99949**	0,99872
	DE	0,99885	**0,99948**	**0,99996**	0,99948	0,99556	0,99996	**0,99877**	0,99948	**0,99994**
	PSO	0,99942	**0,99956**	**0,99997**	0,99892	0,99348	0,99998	**0,99881**	**0,99949**	**0,99942**
	FPA	0,99900	**0,99957**	**0,99997**	**0,99976**	0,99850	0,99995	0,99858	**0,99949**	**0,99994**
Precision	SS	**0,99807**	0,99292	0,99692	0,99990	**0,99514**	0,99989	0,99446	0,99895	0,99058
	DE	0,99537	**0,99998**	**0,99984**	**0,99998**	0,97134	0,99969	**0,99458**	**0,99987**	**0,99967**
	PSO	0,99574	**0,99999**	**0,99988**	0,99599	0,95410	0,99988	**0,99496**	**0,99954**	**0,99573**
	FPA	0,99646	**0,99995**	**0,99989**	**0,99999**	0,99277	**0,99999**	0,99329	0,99887	**0,99967**
Recall	SS	0,99968	0,99987	0,99988	0,99679	0,99614	0,99998	0,99629	**0,99614**	0,99685
	DE	0,99700	**0,99989**	**0,99989**	0,99614	**0,99825**	**0,99998**	**0,99721**	0,99614	**0,99927**
	PSO	**0,99975**	0,99915	**0,99997**	0,99594	**0,99784**	**0,99999**	**0,99981**	0,99614	**0,99844**
	FPA	0,99860	0,99984	**0,99988**	**0,99819**	**0,99967**	0,99995	**0,99743**	0,99614	**0,99958**
F1 Score	SS	**0,99887**	0,99636	0,99839	0,99839	**0,99562**	**0,99994**	0,99536	**0,99807**	0,99524
	DE	0,99569	**0,99803**	**0,99986**	0,99805	0,98352	0,99984	**0,99540**	0,99805	**0,99979**
	PSO	0,99782	**0,99834**	**0,99989**	0,99595	0,97627	0,99994	**0,99553**	0,99807	**0,99784**
	FPA	0,99625	**0,99838**	**0,99987**	**0,99909**	0,99439	0,99983	0,99469	0,99807	**0,99976**

Table 5 Comparative results by runtime and metrics

Studies of Literature									Proposed Study						
References	Datasets	FS	CLF	Evolution Metrics					FS	CLF	Evolution Metrics				
				time	*acc*	*pre*	*rec*	*f1*			*time*	*acc*	*pre*	*rec*	*f1*
[9]	KDDCUP-99	DE	NN	-	0,957	-	-	-	DE	NN	-	**0,98560**	-	-	-
[10]	NSL-KDD	DE	DT	-	0,812	0,74	0,82	0,77	DE	DT	-	**0,89124**	**0,96883**	**0,94967**	**0,90603**
[17]	NSL-KDD	-	MT	-	0,852	-	-	-	FPA	DT	-	**0,91075**	-	-	-
[14]	NSL-KDD	PSO/GA	RF/B	-	0,858	-	-	-	DE	GB	-	**0,96713**	-	-	-
	UNSW-NB15		RF/B	-	0,913	-	-	-	DE	DT	-	**0,92444**	-	-	-
[16]	CSE-CIC-IDS2018	-	ANN	-	0,9997	-	-	-	FPA	RF	-	0,99997	-	-	-
[15]	UNSW-NB15	RFE/RF	LR	-	0,7721	-	-	-	DE	LR	-	**0,90021**	-	-	-
			GB	-	0,6183	-	-	-	FPA	GB	-	**0,92403**	-	-	-
[19]	CSE-CIC-IDS2018	-	CNN	-	0,99	-	-	-	DE/PSO/FPA	ALL CLF	-	0,99990	-	-	-
[20]	KDDCUP-99	PIO	DT	-	0,947	-	-	-	DE/PSO/FPA	DT	-	**0,98000**	-	-	-
	NSL-KDD		DT	-	0,869	-	-	-	DE/PSO/FPA	DT	-	**0,891235/ 0,895759/ 0,910753**	-	-	-
	UNSW-NB15		DT	-	0,913	-	-	-	DE	DT	-	**0,92444**	-	-	-
[18]	NSL-KDD	CFSBA	EC	36,28	0,998	0,998	0,998	0,998	FPA	RF	**2,59,195**	0,98789	0,98948	0,99003	0,98936
[26]	UNSW-NB15	PCA	RF	-	**0,951**	0,948	**0,957**	0,951	FPA	RF	-	0,94856	**0,96334**	0,94948	**0,95240**
			DT	-	**0,947**	**0,944**	0,954	0,948	DE	DT	-	0,92444	0,91301	**0,99978**	**0,95066**
			LR	-	0,682	0,578	0,58	0,548	DE	LR	-	**0,90021**	**0,90424**	**0,97977**	**0,93506**
			ANN	-	0,776	0,762	0,706	0,715	DE	ANN	-	**0,92029**	**0,91631**	**0,99088**	**0,94757**
[28]	CSE-CIC-IDS2018	FPA	LR	20,578	0,946	0,971	0,964	0,968	FPA	LR	**26,395**	**0,99900**	**0,99646**	**0,99860**	**0,99625**
			AB	593,7	0,862	0,961	0,883	0,92	FPA	AB	**432,353**	**0,99995**	**0,99999**	**0,99995**	**0,99983**
			RF	445,1	0,982	0,991	0,988	0,989	FPA	RF	**97,441**	**0,99997**	**0,99989**	**0,99988**	**0,99987**

**FS* Feature Selection, *CLF* Classifier, *RFE/RF* Recursive Feature Elimination/RF, *PIO,* Pigeon-Inspired Optimization, *CFSBA* Correlation Based Feature Selection Bat Algorithm, *MT* Multi-Tree, *RF/B* Rotation Forest/Bagging, *EC,* Ensemble Classifier. Conclusion and Recommendations

to the proposed work are also considered to make a fair comparison. Looking at the overall outputs in Table 5, it can be observed that the model proposed in this study is more successful when compared to the studies in the literature. For instance, when comparing the results of Salek et al., who applied the NN classifier with the DE feature selection approach on the KDD CUP 99 dataset, to those of the proposed model, the latter demonstrates superior performance, achieving an accuracy of 0.9856 [9]. When the proposed study is compared with the study of Zhou et al., which is also a different study, it is seen that there are close results in terms of performance metrics [18]. However, compared to time, this study was found to be more successful by a large margin. In Table 5, the results of the studies in the literature and the proposed model are compared, and the better results in terms of time and performance metrics are indicated in bold. Overall, the results indicate that the framework proposed in this study delivers successful outcomes.

6 Conclusion and Recommendations

Cyber security is seen as an important area in combating cyber-attacks and reducing related costs and damages. The protection of information in terms of the principles of confidentiality, integrity, and accessibility, which are the basic components of cyber security, is of great importance. Intrusion Detection Systems, an essential component of the security architecture in terms of cyber security, serve as a tool that recognizes various types of intrusions and generates alerts. An IDS monitors a network and identifies suspicious and malicious activities or policy violations through network traffic, which allows network administrators to continuously monitor current threats. The main task of IDSs is to protect a system by applying various combinations of actions to prevent intruders or malicious activities that compromise system security. Anomaly-based IDSs build a model of the system's normal behavior, looking for activities that deviate from the model to identify attacks based on irregularities in the model. Anomaly-based CSSs are characterized by their ability to detect new and unknown attacks, i.e., new/zero-day attack types. However, the emergence of a higher false positive rate limits the application of anomaly-based IDSs. The list of threats in the network and on computers is endless and constantly evolving; therefore, new/zero-day attack types are increasing every day. Anomaly-based IDSs continue to be an active area of research, focusing on anomaly-based approaches that use machine learning techniques, i.e., artificial intelligence methodology, to identify intrusions. In order to improve the performance of intrusion detection systems, current research has adopted the use of heuristic algorithms in combination with machine learning algorithms for the selection of features that are effective in intrusion prediction. This study proposes a framework that employs FPA, PSO, and DE methods for feature selection, utilizing logistic regression, decision tree, random forest, linear discriminant analysis, quadratic discriminant analysis, AdaBoost, k-nearest neighbor, naive Bayes, gradient boosting, and neural network classifiers. The framework is evaluated using the KDD CUP 99, NSL-KDD, UNSW-NB15, and CICIDS2018 datasets. The outputs of the models are analyzed by comparing the results both without feature selection and with feature selection-based approaches from the literature.

The analysis results in this study show that the model's performance based on feature selection is about 200% better than the classification performed by machine learning algorithms alone without feature selection. While all algorithms improved performance in terms of runtime, the results of the evaluation criteria related to intrusion detection capabilities showed that the deficiencies found in the data sets were directly proportional to the classification result. When the results are analyzed, it is summarized that successful outputs are obtained using feature selection approaches. Accordingly, in KDD CUP 99; DE method 0.985497, 0.964215, 0.971441, 0.95633, 0.995529 in DT, QDA, LDA, NB, GB respectively; PSO method 0.985514, 0.996597, 0.971896, 0.980596, 0.994985 in DT, RF, QDA, LDA, GB respectively; FPA method achieved the best accuracy values of 0.984062, 0.980887, 0.979798, 0.991675 in DT, LDA, NB, GB, respectively. In NSL-KDD, which is used as the second dataset, the best accuracy results were obtained as follows: DE approach 0.891235, 0.967131, 0.895759, 0.852156, 0.910753, 0.98789 in DT, GB; PSO approach 0.895759, 0.852156 in DT, QDA; FPA approach 0.910753, 0.98789 in DT, RF, respectively. In the third dataset, UNSW-NB15, the best accuracy outputs were 0.900211, 0.924438, 0.861806, 0.861806, 0.893499, 0.893499, 0.920286, 0.841891 in DE method LR, DT, QDA, LDA, NN, NB respectively; 0.896758 in PSO method DT, 0.867061 in FPA method DT. Finally, in the experimental studies on the CSE-CIC-IDS2018 dataset, the DE approach achieved 0.999476, 0.999964, 0.998773, 0.999944 in DT, RF, NN, GB, respectively,PSO approach achieved the best accuracy values of 0.999557, 0.998807, 0.999419 in DT, NN, GB respectively; FPA approach achieved the best accuracy values of 0.999568, 0.999965, 0.999758, 0.999937 in DT, RF, QDA, and GB, respectively.

Building on the findings of this study, it is clear that feature selection approaches will play a pivotal role in shaping the success of future research in IDS. A critical insight is the direct influence of dataset generation processes on prediction accuracy and IDS performance. Future work should prioritize refining these processes to enhance IDS effectiveness. As network traffic continues to evolve, particularly in real-time monitoring, the volume of traffic features will inevitably exceed current expectations due to the dynamic nature of

modern networks. Leveraging feature selection methods to isolate and retain the most impactful features will be key to optimizing the performance of anomaly-based IDSs. In conclusion, these findings offer valuable insights that will drive the development of more efficient, real-time IDS monitoring systems, fostering a deeper understanding and advancement of anomaly detection techniques in increasingly complex network environments.

Future work should explore advanced hybrid models that integrate deep learning with heuristic feature selection to further optimize IDS performance. Additionally, the development of lightweight and adaptive IDS solutions for real-time applications remains a critical direction. Investigating the impact of feature selection in multi-class classification scenarios and expanding analyses to newer, more complex datasets will also contribute to advancing anomaly detection techniques in dynamic network environments.

Acknowledgements This study was supported by the Scientific Research Projects Coordination Unit (BAP) of Gazi University under grant number FGA-2022-8043.

Author Contributions In this study, Emre was responsible for data collection, preprocessing, writing the code, and conducting the experiments. Dr. Yılmaz managed the overall project, performed code analysis, and took responsibility for interpreting and evaluating the test results. Both authors contributed equally to the conceptualization and execution of the research, with Emre focusing on technical implementation and Dr. Yılmaz overseeing the project and ensuring the scientific integrity of the study.

Funding Open access funding provided by the Scientific and Technological Research Council of Türkiye (TÜBİTAK). No financial support was received for this study.

Data availability No datasets were generated or analysed during the current study.

Declarations

This is to confirm that this manuscript has not been published elsewhere and is not under consideration by any other journal. All the authors have approved the manuscript with full text, and numbers of figures/tables and agree with its submission to the journal.

Competing interest The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Cisco Visual Networking Index (VNI) Global mobile data traffic forecast update, 2017. Available online: https://newsroom.cisco.com/c/r/newsroom/en/us/a/y2017/m02/cisco-mobile-visual-networking-index-vni-forecast-projects-7-fold-increase-in-global-mobile-data-traffic-from-2016-2021.html. Accessed 21 June 2024
2. Daş R, Baykara M (2019) Saldırı tespit ve engelleme araçlarının incelenmesi. Dicle Üniversitesi Mühendislik Fakültesi Mühendislik Dergisi 10(1):57–75. https://doi.org/10.24012/dumf.449059
3. He K, Kim DD, Asghar MR (2023) Adversarial machine learning for network intrusion detection systems: a comprehensive survey. IEEE Commun Surv Tutorials 25(1):538–566. https://doi.org/10.1109/COMST.2022.3233793
4. Rao UH, Nayak U (2014) Intrusion detection and prevention systems. In: The InfoSec Handbook: An Introduction to Information Security (pp. 225–243). Apress. https://doi.org/10.1007/978-1-4302-6383-8_11
5. Akhiat Y, Touchanti K, Zinedine A, Chahhou M (2024) IDS-EFS: Ensemble feature selection-based method for intrusion detection system. Multimedia Tools Appl 83(5):12917–12937. https://doi.org/10.1007/s11042-023-15977-8
6. Latif N, Ma W, Ahmad HB (2025) Advancements in securing federated learning with IDS: a comprehensive review of neural networks and feature engineering techniques for malicious client detection. Artif Intell Rev 58(3):91. https://doi.org/10.1007/s10462-024-11082-w
7. Bakro M, Kumar RR, Husain M, Ashraf Z, Ali A, Yaqoob SI, Ahmed MN, Parveen N (2024) Building a Cloud-IDS by hybrid bio-inspired feature selection algorithms along with random forest model. IEEE Access 12:8846–8874. https://doi.org/10.1109/ACCESS.2024.3353055
8. Deore B, Bhosale S (2023) Adaptive dolphin atom search optimization-based DRNN for network intrusion detection system. SN Computer Science 4(5):578. https://doi.org/10.1007/s42979-023-02006-6
9. Salek Z, Madani FM, Azmi R (2013) Intrusion detection using neuarl networks trained by differential evaluation algorithm. 2013 10th International ISC Conference on Information Security and Cryptology (ISCISC), 1–6. https://doi.org/10.1109/ISCISC.2013.6767341
10. Popoola E, Adewumi AO (2017) Efficient feature selection technique for network intrusion detection system using discrete differential evolution and decision. Int J Netw Secur 19(5):660–669
11. Hajisalem V, Babaie S (2018) A hybrid intrusion detection system based on ABC-AFS algorithm for misuse and anomaly detection. Comput Netw 136:37–50. https://doi.org/10.1016/j.comnet.2018.02.028
12. Selvakumar B, Muneeswaran K (2018) Firefly algorithm based feature selection for network intrusion detection. Comput Secur 81:148–155. https://doi.org/10.1016/j.cose.2018.11.005
13. Mazini M, Shirazi B, Mahdavi I (2018) Anomaly network-based intrusion detection system using a reliable hybrid artificial bee colony and AdaBoost algorithms. https://doi.org/10.1016/j.jksuci.2018.03.011
14. Tama BA, Comuzzi M, Rhee K (2019) TSE-IDS: A Two-Stage Classifier Ensemble for Intelligent Anomaly-Based Intrusion Detection System. https://doi.org/10.1109/ACCESS.2019.2928048
15. Meftah S, Rachidi T, Assem N (2019) Network based intrusion detection using the UNSW-NB15 dataset. Int J Comput Digital Syst 8(5):2210–142. https://doi.org/10.12785/ijcds/080505
16. Kanimozhi V, Jacob TP (2019) Artificial intelligence based network intrusion detection with hyper-parameter optimization tuning on the realistic cyber dataset CSE-CIC-IDS2018 using cloud

computing. ICT Express 5(3):211–214. https://doi.org/10.1016/J.ICTE.2019.03.003
17. Gao X, Shan C, Hu C, Niu Z, Liu Z (2019) An adaptive ensemble machine learning model for intrusion detection. IEEE Access 7:82512–82521. https://doi.org/10.1109/ACCESS.2019.2923640
18. Zhou Y, Cheng G, Jiang S, Dai M (2020) Building an efficient intrusion detection system based on feature selection and ensemble classifier. Comput Netw 174:107247. https://doi.org/10.1016/j.comnet.2020.107247
19. Kim J, Kim J, Kim H, Shim M, Choi E (2020) CNN-based network intrusion detection against denial-of-service attacks. https://doi.org/10.3390/electronics9060916
20. Alazzam H, Sharieh A, Sabri KE (2020) A feature selection algorithm for intrusion detection system based on Pigeon Inspired Optimizer. Expert Syst Appl 148:113249. https://doi.org/10.1016/j.eswa.2020.113249
21. Devan P, Khare N (2020) An efficient XGBoost-DNN-based classification model for network intrusion detection system. Neural Comput Appl 32. https://doi.org/10.1007/s00521-020-04708-x
22. Nazir A, Ahmed Khan R (2021) A novel combinatorial optimization based feature selection method for network intrusion detection. https://doi.org/10.1016/j.cose.2020.102164
23. Liu J, Gao Y, Hu F (2021) A fast network intrusion detection system using adaptive synthetic oversampling and LightGBM. Comput Secur 106:102289. https://doi.org/10.1016/j.cose.2021.102289
24. Thilagam T, Aruna R (2021) Intrusion detection for network based cloud computing by custom RC-NN and optimization-NC-ND license. https://doi.org/10.1016/j.icte.2021.04.006. http://creativecommons.org/licenses/bync-nd/4.0/. Accessed 24 Jul 2024
25. Disha RA, Waheed S (2022) Performance analysis of machine learning models for intrusion detection system using Gini Impurity-based Weighted Random Forest (GIWRF) feature selection technique 5(1). https://doi.org/10.1186/s42400-021-00103-8
26. Ahmad I, Haq QEU, Imran M, Alassafi MO, Alghamdi RA (2022) An efficient network intrusion detection and classification system. Mathematics 10(3):530. https://doi.org/10.3390/MATH10030530
27. Qureshi A-U-H, Larijani H, Ahmad J, Mtetwa N (2018) A novel random neural network based approach for intrusion detection systems. In: 2018 10th Computer science and electronic engineering (CEEC). Colchester, UK, pp 50–55. https://doi.org/10.1109/CEEC.2018.8674228
28. NajafiMohsenabad H, Tut MA (2024) Optimizing cybersecurity attack detection in computer networks: a comparative analysis of bio-inspired optimization algorithms using the CSE-CIC-IDS 2018 dataset. Applied Sciences 14(3):1044. https://doi.org/10.3390/APP14031044
29. Zare F, Mahmoudi-Nasr P, Yousefpour R (2024) A real-time network based anomaly detection in industrial control systems. Int J Crit Infrastruct Prot 45:100676. https://doi.org/10.1016/j.ijcip.2024.100676
30. Saheed YK, Abdulganiyu OH, Majikumna KU, Mustapha M, Workneh AD (2024) ResNet50-1D-CNN: A new lightweight resNet50-One-dimensional convolution neural network transfer learning-based approach for improved intrusion detection in cyber-physical systems. Int J Crit Infrastruct Prot 45:100674. https://doi.org/10.1016/j.ijcip.2024.100674
31. Habeeb MS, Babu TR (2024) Coarse and fine feature selection for Network Intrusion Detection Systems (IDS) in IoT networks. Transactions on Emerging Telecommunications Technologies, 35(4). https://doi.org/10.1002/ETT.4961
32. Yuan X, Han S, Huang W, Ye H, Kong X, Zhang F (2024) A simple framework to enhance the adversarial robustness of deep learning-based intrusion detection system. Comput Secur 137:103644. https://doi.org/10.1016/J.COSE.2023.103644
33. Truong VT, Le LB (2024) Security for the metaverse: blockchain and machine learning techniques for intrusion detection. IEEE Network. https://doi.org/10.1109/MNET.2024.3351882
34. Gupta B, Agrawal DP, Yamaguchi S (eds) (2016) Handbook of Research on Modern Cryptographic Solutions for Computer and Cyber Security. IGI Global. https://doi.org/10.4018/978-1-5225-0105-3
35. Vasilomanolakis E, Karuppayah S, Muhlhauser M, Fischer M (2015) Taxonomy and survey of collaborative intrusion detection. ACM Computing Surveys (CSUR) 47(4). https://doi.org/10.1145/2716260
36. Hugh JM (2000) Testing intrusion detection systems: a critique of the 1998 and 1999 DARPA Intrusion Detection System Evaluations as Performed by Lincoln Laboratory.
37. Brown C, Cowperthwaite A, Hijazi A, Somayaji A (2009) Analysis of the 1999 DARPA/Lincoln Laboratory IDS evaluation data with NetADHICT; Analysis of the 1999 DARPA/Lincoln Laboratory IDS evaluation data with NetADHICT. https://doi.org/10.1109/CISDA.2009.5356522
38. KDD Cup 1999 Data (n.d.). https://kdd.ics.uci.edu/databases/kddcup99/kddcup99. Accessed 21 May 2024
39. Stolfo SJ, Fan W, Lee W, Prodromidis A, Chan PK (2000) Cost-based modeling for fraud and intrusion detection: results from the JAM Project. http://www.cs.columbia.edu/. Accessed 14 Sept 2024
40. Gogoi P, Bhuyan MH, Bhattacharyya DK, Kalita JK (2012) Packet and flow based network intrusion dataset. In: Parashar M, Kaushik D, Rana OF, Samtaney R, Yang Y, Zomaya A (eds) Contemporary computing. Springer Berlin Heidelberg, pp 322–334
41. The UNSW-NB15 Dataset | UNSW Research (2024). https://research.unsw.edu.au/projects/unsw-nb15-dataset. Accessed 28 May 2024
42. IDS 2018 | Datasets | Research | Canadian Institute for Cybersecurity | UNB (n.d.). https://www.unb.ca/cic/datasets/ids-2018.html. Accessed 28 May 2024
43. Sharafaldin I, Lashkari AH, Ghorbani AA (2018) Toward generating a new intrusion detection dataset and intrusion traffic characterization. https://doi.org/10.5220/0006639801080116
44. NSL-KDD | Datasets | Research | Canadian Institute for Cybersecurity | UNB. (n.d.). from https://www.unb.ca/cic/datasets/nsl.html. Accessed 16 Apr 2024
45. Kennedy J (2010) Particle Swarm Optimization. In: G. I. Sammut Claude and Webb (ed) Encyclopedia of Machine Learning (pp. 760–766). Springer US. https://doi.org/10.1007/978-0-387-30164-8_630
46. Zhang Y, Wang S, Ji G (2015) A comprehensive survey on particle swarm optimization algorithm and its applications. Math Problems Eng. https://doi.org/10.1155/2015/931256
47. Kumar D, Ramakrishnan AG (2016) Binary classification posed as a quadratically constrained quadratic programming and solved using particle swarm optimization. Sadhana - Academy Proc Eng Sci 41(3):289–298. https://doi.org/10.1007/S12046-016-0466-Y/TABLES/7
48. Mirjalili S, Wang GG, Coelho L dos S (2014). Binary optimization using hybrid particle swarm optimization and gravitational search algorithm. Neural Comput Appl 25(6):1423–1435. https://doi.org/10.1007/S00521-014-1629-6/FIGURES/13
49. Cronquist A (1981) An integrated system of classification of flowering plants. Columbia University Press, New York, pp 248–250
50. Hutchings MJ, Bell AD (1991) Plant form: an illustrated guide to flowering plant morphology. J Ecol 79(2):557
51. Glover B (2007) Understanding flowers and flowering: an integrated approach, 1st edn. Oxford Academic, Oxford. https://doi.org/10.1093/acprof:oso/9780198565970.001.0001

52. Kalra S, Arora S (2016) Firefly algorithm hybridized with flower pollination algorithm for multimodal functions. In: Proceedings of the international congress on information and communication technology: ICICT 2015 1. pp 207–219
53. Pavlyukevich I (2007) Lévy flights, non-local search and simulated annealing. J Comput Phys 226(2):1830–1844
54. Yang X-S (2012) Flower pollination algorithm for global optimization. In: Durand-Lose J, Jonoska N (eds) Unconventional computation and natural computation. Springer Berlin Heidelberg, pp 240–249
55. Storn R, Price K (1997) Differential evolution - a simple and efficient heuristic for global optimization over continuous spaces. J Global Optim 11(4):341–359. https://doi.org/10.1023/A:1008202821328/METRICS
56. Ahmad Z, Shahid Khan A, Wai Shiang C, Abdullah J, Ahmad F (2020) Network intrusion detection system: A systematic study of machine learning and deep learning approaches. Trans Emerging Telecommun Technol 32(1). https://doi.org/10.1002/ETT.4150
57. Miguel-Hurtado O, Guest R, Stevenage SV, Neil GJ, Black S (2016) Comparing machine learning classifiers and linear/logistic regression to explore the relationship between hand dimensions and demographic characteristics. PLoS ONE 11(11):e0165521
58. Ng A, Jordan M (2001) On discriminative vs. generative classifiers: a comparison of logistic regression and naive bayes. Adv Neural Inf Process Syst 14
59. Rokach L, Maimon O (2005) Decision Trees. Data Mining and Knowledge Discovery Handbook, 165–192. https://doi.org/10.1007/0-387-25465-X_9
60. Song YY, Lu Y (2015) Decision tree methods: applications for classification and prediction. Shanghai Arch Psychiatry 27(2):130. https://doi.org/10.11919/J.ISSN.1002-0829.215044
61. Wu D, Jennings C, Terpenny J, Gao RX, Kumara S (2017) A comparative study on machine learning algorithms for smart manufacturing: tool wear prediction using random forests. J Manuf Sci Eng, Trans ASME 139(7). https://doi.org/10.1115/1.4036350/454654
62. Subasi A (2020) Practical machine learning for data analysis using python. https://doi.org/10.1016/B978-0-12-821379-7.00008-4
63. Chen H, Wu L, Chen J, Lu W, Ding J (2022) A comparative study of automated legal text classification using random forests and deep learning ☆. Inf Process Manage 59:102798. https://doi.org/10.1016/j.ipm.2021.102798
64. Asharf J, Moustafa N, Khurshid H, Debie E, Haider W, Wahab A (2020) Electronics a review of intrusion detection systems using machine and deep learning in internet of things: challenges. Solutions Future Dir. https://doi.org/10.3390/electronics9071177
65. Liu G, Zhao H, Fan F, Liu G, Xu Q, Nazir S (2022) An enhanced intrusion detection model based on improved kNN in WSNs. Sensors 22(4):1407. https://doi.org/10.3390/S22041407
66. Shafer G (1985) Conditional Probability. Int Stat Rev Revue Internationale de Statistique 53(3):261. https://doi.org/10.2307/1402890
67. Sharmila BS, Nagapadma R (2019) Intrusion detection system using naive bayes algorithm. 2019 5th IEEE International WIE Conference on Electrical and Computer Engineering, WIECONECE 2019 - Proceedings. https://doi.org/10.1109/WIECONECE48653.2019.9019921
68. Dadkhah S, Mahdikhani H, Danso PK, Zohourian A, Truong KA, Ghorbani AA (n.d.) Towards the development of a realistic multidimensional IoT profiling dataset. https://doi.org/10.1109/PST55820.2022.9851966
69. Xanthopoulos P, Pardalos PM, Trafalis TB (2013) Linear discriminant analysis. In: Robust Data Mining (pp. 27–33). Springer New York. https://doi.org/10.1007/978-1-4419-9878-1_4
70. Lee H (2016) Clustering and classification methods for prediction of the risk for developing disease. Doctoral Dissertation, State University of New York at Stony Brook, ProQuest Dissertations & Theses, 10165402
71. Abdulhammed R, Musafer H, Alessa A, Faezipour M, Abuzneid A (2019) Features Dimensionality Reduction Approaches for Machine Learning Based Network Intrusion Detection. Electronics 8(3):322. https://doi.org/10.3390/ELECTRONICS8030322
72. Peppes N, Daskalakis E, Alexakis T, Adamopoulou E, Demestichas K (2021) Performance of machine learning-based multi-model voting ensemble methods for network threat detection in agriculture 4.0. Sensors 21(22):7475. https://doi.org/10.3390/S21227475
73. McCulloch WS, Pitts W (1943) A logical calculus of the ideas immanent in nervous activity. Bull Math Biophys 5:115–133
74. Hodo E, Bellekens X, Hamilton A, Dubouilh PL, Iorkyase E, Tachtatzis C, Atkinson R (2016) Threat analysis of IoT networks using artificial neural network intrusion detection system. 2016 International Symposium on Networks, Computers and Communications, ISNCC 2016. https://doi.org/10.1109/ISNCC.2016.7746067
75. Li P, Chen Z, Yang LT, Gao J, Zhang Q, Deen MJ (2019) An incremental deep convolutional computation model for feature learning on industrial big data. IEEE Trans Industr Inf 15(3):1341–1349. https://doi.org/10.1109/TII.2018.2871084
76. Goodfellow I, Bengio Y, Courville A (2016) Deep learning. MIT press

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Emre Emirmahmutoğlu completed his bachelor's degree from Department of Electrical and Electronics Engineering, Ankara Yıldırım Beyazıt University in 2018. He started his career in the field of information technologies. He received his master's degree from Department of Cyber Security, Alparslan Defence Sciences and National Security Institute, National Defence University in 2024. His main research focus is in the areas of cyber security, information technology, information security, machine learning, and artificial ıntelligence. He is currently working as a System Solutions Senior Specialist at Netaş Information Technologies in Ankara, Türkiye.



Dr. Yılmaz Atay received his BSc, MSc, and PhD degrees in Computer Engineering from Selcuk University, Konya, Türkiye, in 2010, 2012, and 2018, respectively. Between 2012 and 2018, he served as a research assistant at the same institution. In 2016, he conducted research at the University of Florida. Since 2020, Dr. Atay has been a faculty member at Gazi University, Ankara, Türkiye. His research interests include machine learning, data mining, bioinformatics, complex network analysis, and graph-based approaches.

Authors and Affiliations

Emre Emirmahmutoğlu[1] · Yılmaz Atay[2]

✉ Yılmaz Atay
yilmazatay@gazi.edu.tr

Emre Emirmahmutoğlu
emreemirmahmutoglu@gmail.com

[1] Department of Cyber Security, Alparslan Defence Sciences and National Security Institute, National Defence University, Ankara, Turkey

[2] Department of Computer Engineering, Faculty of Engineering, Gazi University, Ankara, Turkey